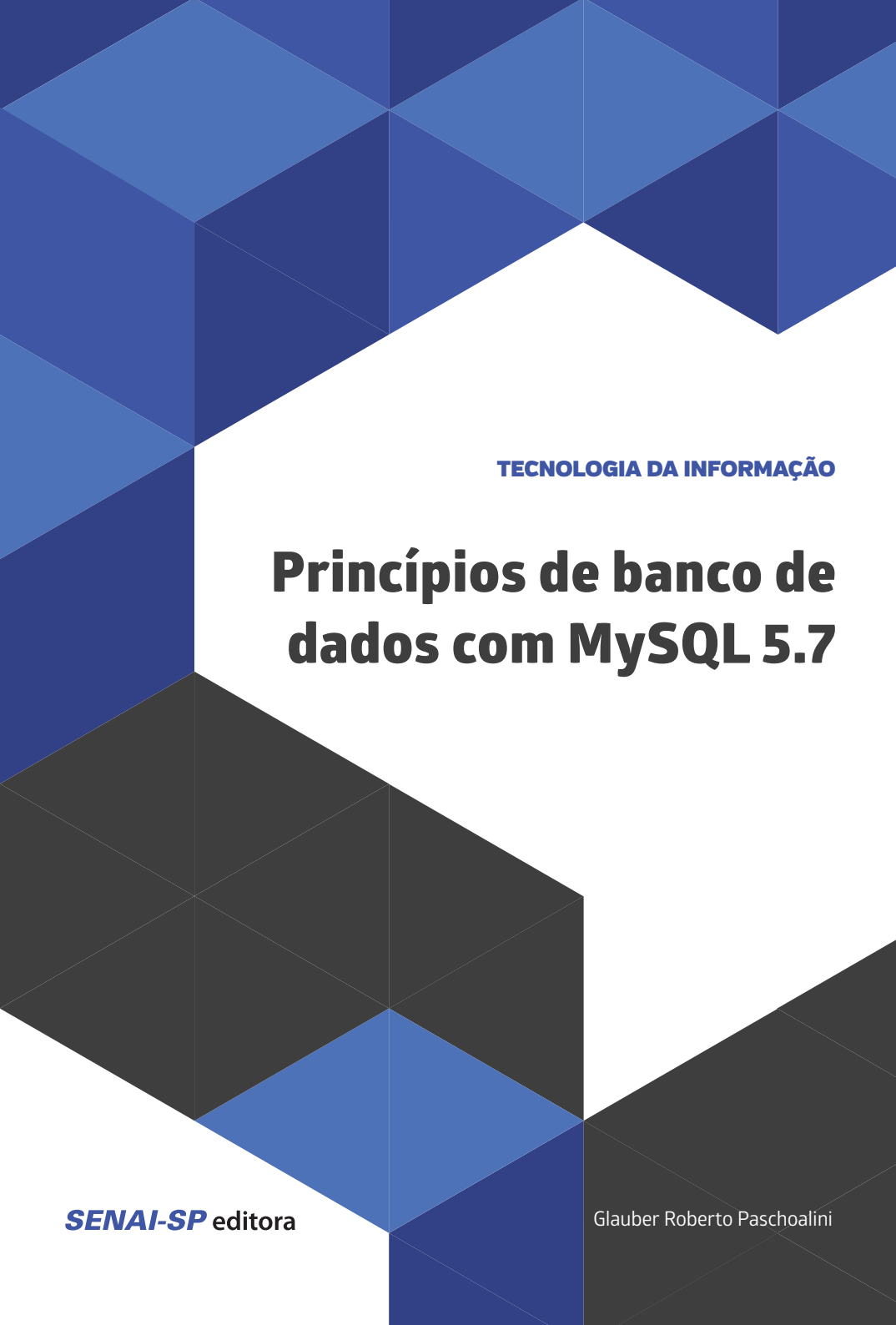




TECNOLOGIA DA INFORMAÇÃO

Princípios de banco de dados com MySQL 5.7

SENAI-SP editora

Glauber Roberto Paschoalini

Princípios de banco de dados com MySQL 5.7

Dados Internacionais de Catalogação na Publicação (CIP)

Paschoalini, Glauber Roberto

Princípios de banco de dados com MySQL 5.7 / Glauber Roberto

Paschoalini – São Paulo : SENAI-SP Editora, 2019.

144 p. : il.

Inclui referências

ISBN 978-85-8393-992-4

1. MySQL (Recurso eletrônico). 2. Banco de dados - Projeto.
3. Modelagem de dados. I. Título.

CDD 005.133

Índice para o catálogo sistemático:

1. MySQL (Recurso eletrônico) 005.133

Biblioteca responsável: Vera Lúcia Piorno Barbosa CRB-8 3279

SENAI-SP Editora

Avenida Paulista, 1313, andar intermediário, 01311 923, São Paulo – SP

F. 11 3146.7308 | editora@sesisenaisp.org.br

www.senaispeditora.com.br

TECNOLOGIA DA INFORMAÇÃO

Princípios de banco de dados com MySQL 5.7

Glauber Roberto Paschoalini

SENAI-SP editora



**Departamento Regional
de São Paulo**

Presidente

Josué Christiano Gomes da Silva

Diretor Regional

Ricardo Figueiredo Terra

*Diretoria Corporativa e
de Estratégias Educacionais*

Wilson Risolia Rodrigues

*Gerência de Assistência
à Empresa e à Comunidade*

Celso Taborda Kopp

Gerência de Inovação e de Tecnologia

Osvaldo Lahoz Maia

Gerência de Educação

Cassia Regina Souza da Cruz

Apresentação

Com a permanente transformação dos processos produtivos e das formas de organização do trabalho, as demandas por educação profissional multiplicam-se e, sobretudo, diversificam-se.

Em sintonia com essa realidade, o SENAI-SP valoriza a educação profissional para o primeiro emprego, dirigida a jovens. Privilegia também a qualificação de adultos que buscam um diferencial de qualidade para progredir no mercado de trabalho. E incorpora firmemente o conceito de “educação ao longo de toda a vida”, oferecendo modalidades de formação continuada para profissionais já atuantes. Dessa forma, atende às prioridades estratégicas da Indústria e às prioridades sociais do mercado de trabalho.

A instituição trabalha com cursos de longa duração como os cursos de Aprendizagem Industrial, os cursos Técnicos e os cursos Superiores de Tecnologia. Oferece também cursos de Formação Inicial e Continuada, com duração variada nas modalidades de Iniciação Profissional, Qualificação Profissional, Especialização Profissional, Aperfeiçoamento Profissional e Pós-Graduação.

Com satisfação, apresentamos ao leitor esta publicação, que integra uma série da SENAI-SP Editora, especialmente criada para apoiar os alunos das diversas modalidades.

Sumário

Introdução	9
1. Modelagem de dados	13
Banco de dados	14
Modelo entidade-relacionamento	17
Entidade	18
Atributo de entidade	18
Modelo conceitual	19
Modelo lógico	26
Modelo físico	30
2. Linguagem de Definição de Dados (DDL)	33
Comando SHOW	35
Comando CREATE DATABASE	36
Comando DROP DATABASE	37
Comando USE	39
Comando CREATE TABLE	40
Comando DROP TABLE	47
Comando ALTER TABLE	48
3. Linguagem de Manipulação de Dados (DML)	52
Comando INSERT	53
Comando SELECT	55
Comando DELETE	69
Comando UPDATE	70
4. Funções	74
Funções matemáticas	75
Funções de <i>string</i>	76
Funções de data e hora	78
Funções de agregação	80
Subconsulta	82

5. Consultas com informações agrupadas	85
Cláusula GROUP BY	86
Cláusula HAVING	88
Cláusula DISTINCT	89
Operador UNION	90
6. Junção de tabelas	94
Relação entre tabelas	95
Cláusula INNER JOIN	98
Cláusula LEFT JOIN	99
Cláusula RIGHT JOIN	101
7. Índices e visões	104
Comando CREATE VIEW	105
Comando ALTER VIEW	106
Comando DROP VIEW	107
Comando CREATE INDEX	107
Comando DROP INDEX	108
8. Funções, procedimentos e gatilhos	111
Funções	112
Programação de funções	117
Procedimentos	126
Gatilhos	131
Considerações finais	137
Referências	139
Sobre o autor	141

Introdução

O MySQL é um banco de dados completo, robusto e extremamente rápido, com todas as características existentes nos principais bancos de dados disponíveis no mercado. Seu ponto alto é a licença gratuita para fins estudantis e realização de negócios.

Estruturado didaticamente, este livro apresenta os recursos de um Sistema Gerenciador de Banco de Dados (SGBD) para a criação e manutenção de informações que serão utilizadas por um programa de computador.

É importante ter conhecimento de lógica de programação, principalmente para os capítulos que tratam de funções e procedimentos.

Os principais assuntos tratados são:

- modelagem de dados;
- comandos da Linguagem de Definição de Dados;
- comandos da Linguagem de Manipulação de Dados;
- linguagem de programação para banco de dados;
- *views* e *triggers*.

Após o levantamento de requisitos, utiliza-se a modelagem de dados para compreender em detalhes as regras do negócio por meio de modelos reconhecidos na área de informática, o que facilita o trabalho do desenvolvedor e de outros membros da equipe que terão acesso aos documentos.

A Linguagem de Definição de Dados, cuja sigla em inglês é DDL (*Data Definition Language*), contém os comandos para a criação de usuários, banco de dados e tabelas, e faz parte da última fase da modelagem de dados, conhecida como modelo físico.

Já a Linguagem de Manipulação de Dados, cuja sigla em inglês é DML (*Data Manipulation Language*), contém os comandos para inclusão, exclusão, alteração e consulta de dados. Este último apresenta diversas variações que permitem selecionar de modo específico o que se deseja extrair de informação do banco. É a parte do banco que mais se aproxima da programação do sistema e, por isso, é de grande importância para alunos de programação e desenvolvedores.

A linguagem de programação de banco de dados permite a criação de funções e procedimentos que maximizem o desempenho de uso do banco por meio de rotinas proprietárias, que serão executadas no próprio banco, e produz respostas internas ou que serão entregues como parte do resultado de consultas.

São abordadas também as *views*, ou visões, e os *triggers*, ou gatilhos, que completam o conhecimento de quem está estabelecendo o primeiro contato com banco de dados.

O MySQL 5.7 Community Server (<http://www.mysql.com>) é um produto da Oracle Corporation Brazil e foi utilizado para o desenvolvimento deste livro. Os requisitos básicos de configuração de *hardware* e *software* são:

- Microsoft Windows 7 ou superior;
- no mínimo 2 GB de memória RAM;
- no mínimo 200 MB de espaço livre no disco rígido;
- suporte ao protocolo TCP/IP;

- instalador do MySQL Community Server para Windows;
- conhecimentos de *prompt* de comandos do Microsoft Windows.

Para outros sistemas operacionais, consultar a documentação oficial em <<https://dev.mysql.com/doc/refman/5.7/en/installing.html>>.

O brModelo 3.0 (<https://sourceforge.net/projects/brmodelo30/>) é uma ferramenta livre e foi utilizada para desenvolver os diagramas, porém, podem ser utilizadas outras ferramentas. A configuração básica necessária para utilizá-la é ter a versão Microsoft Windows 7 ou superior; ter no mínimo 1 GB de memória RAM e no mínimo 5 MB de espaço livre no disco rígido.

Após a leitura dos capítulos e a execução dos exercícios, o aluno estará apto a acrescentar em seus programas o acesso e o uso de banco de dados para manipular grandes quantidades de dados, como é exigido atualmente em quase todas as áreas que utilizam sistemas informatizados.

1. Modelagem de dados

Banco de dados

Modelo entidade-relacionamento

Entidade

Atributo de entidade

Modelo conceitual

Modelo lógico

Modelo físico

Toda atividade que se pretende realizar depende de planejamento.

Às vezes, esse planejamento é simples como selecionar o prato do almoço em um restaurante. Outras vezes, é complexo como definir uma viagem para o período de férias.

Por meio do planejamento, é possível organizar o que será feito para garantir que o objetivo final seja atingido com sucesso.

A modelagem de dados é o planejamento que se executa com a finalidade de garantir que serão armazenadas todas as informações necessárias para manter um sistema funcionando corretamente, sem excesso ou falta de dados.

Com a modelagem de dados, o programador poderá compreender o funcionamento da regra de negócio do sistema que será desenvolvido, ao utilizar um banco de dados para preservar as informações.

O programa **brModelo** será usado para criar diagramas, portanto, a mesma nomenclatura dos componentes de modelagem proposta por ele será adotada.

Este capítulo aborda:

- o que é um banco de dados;
- a importância do Modelo Entidade-Relacionamento (MER);
- as entidades e seus atributos;
- a criação do modelo conceitual a partir do levantamento de requisitos;
- a representação do modelo lógico por meio do Diagrama Entidade-Relacionamento (DER);
- o modelo físico de um banco de dados.

Banco de dados

Para iniciar o estudo de banco de dados, será observado o cenário a seguir, que descreve a regra de negócio de pedidos de uma pizzaria.

Uma pizzaria que entrega em domicílio recebe os pedidos por telefone. O responsável pela empresa precisa garantir que as informações dos clientes estejam sempre atualizadas para assegurar que as entregas ocorram de forma correta.

Quando um cliente liga para a pizzaria, o atendente solicita o seu nome e procura a sua ficha em um fichário para confirmar o endereço de entrega (Figuras 1 e 2).

FICHA DE CLIENTE			
RAZÃO SOCIAL			
INSCRIÇÃO		CNPJ	
ENDEREÇO/CEP			
CIDADE	ESTADO	CEP	TELEFONE/FAX
ENDEREÇO/CEP			
INSCRIÇÃO		CNPJ	

Ivan Sardella

Figura 1 – Ficha de cliente da pizzaria.



Arquivo do autor

Figura 2 – Fichário da pizzaria.

Após confirmar os dados ou abrir uma nova ficha, no caso de novo cliente, o atendente anota a pizza selecionada pelo cliente a partir de um cardápio, calcula o valor total do pedido e confirma a forma de pagamento com o cliente.

O pedido então é levado para a cozinha, as pizzas são preparadas e um entregador as leva para o cliente.

Por razões gerenciais, o proprietário da pizzaria anota, no verso da ficha do cliente, alguns dados de cada pedido, como data e hora em que foi feito o pedido, o valor total a ser pago e a forma de pagamento selecionada pelo cliente.

A ficha do cliente contém diversos dados, como número de telefone e endereço de entrega. O próprio endereço é composto de outros dados, como rua, número e bairro. A pizza possui alguns dados também, como nome e preço.

Dados são fatos, algo cuja existência pode ser constatada de modo indiscutível. No exemplo deste livro, esses dados estão registrados na ficha do cliente.

Os pedidos feitos pelo cliente também contêm dados que foram anotados no verso de sua ficha.

É fundamental que os dados do pedido sejam anotados na ficha correta. Se o cliente “A” pediu uma determinada pizza, qual é a lógica de fazer essa anotação na ficha do cliente “B”?

Entre cliente e pedido, deve haver uma relação que garanta a integridade desses dados. A solução encontrada pela pizzaria foi anotar os pedidos realizados pelo cliente na mesma ficha em que estão seus dados cadastrais.

Quando os dados de uma coleção estão relacionados, como os dados do cliente e dos pedidos, tem-se um banco de dados.

Modelo entidade-relacionamento

Para desenvolver um programa de computador para pizzaria, o primeiro passo é criar um modelo que represente os dados que serão armazenados e a maneira como serão recuperados posteriormente. Essa modelagem contém a especificação das regras de negócio e as estruturas de dados de um banco de dados.

O modelo relacional, mais comum para banco de dados, classifica dados em tabelas compostas por colunas e linhas. Cada coluna lista um atributo da entidade em questão, como preço, código postal ou data de nascimento.

Cada linha, também chamada de tupla, inclui dados sobre uma instância específica da entidade em questão, como um determinado empregado.

Um exemplo de tabelas devidamente relacionadas no caso da pizzaria é ilustrado a seguir. Existem duas tabelas: cliente e pedido (Tabelas 1 e 2, respectivamente). No pedido, não são anotados todos os dados do cliente, mas somente seu código.

Tabela 1 – Códigos dos clientes

Código	Nome
1	Maria
2	João
3	Ana

Tabela 2 – Pedidos

Número	Data	Cliente	Valor total
1	03/01/2017	3	35,00
2	05/01/2017	1	57,00
3	06/01/2017	2	43,00

Dentro do banco de dados, as tabelas podem ser normalizadas ou levadas a cumprir as regras de normalização que tornam o banco de dados flexível, adaptável e escalável. Quando normalizado, cada pedaço de dado é atômico, ou dividido em pequenas peças úteis.

Entidade

Na modelagem do banco de dados, a entidade representa um objeto observado que possui alguns atributos e relaciona-se com outras entidades. Ela pode representar elementos reais, como um cliente, ou abstratos, como um pedido da pizzaria.

Atributo de entidade

Os dados de uma entidade são chamados de atributos na modelagem de dados. Ao determinar os atributos de uma entidade por meio da abstração, deve-se pensar em seu nome e no tipo de informação que representará.

Um tipo específico de atributo merece destaque: o atributo-chave. Esse atributo é usado para identificar a identidade de forma única e, por isso, não pode ter seu valor duplicado. O CPF de uma pessoa é um exemplo de atributo-chave.

Modelo conceitual

O modelo conceitual é o primeiro a ser desenvolvido e normalmente conta com a participação do cliente. Como o nome sugere, trata-se das ideias do novo programa e, portanto, não há relação com qualquer tecnologia. Por essa razão, não há uma forma rígida para a sua representação, que pode ser textual ou gráfica.

O objetivo desta fase da modelagem é determinar quais são as entidades, seus atributos e suas relações, de acordo com a regra de negócios descrita pelo cliente.

No caso da pizzeria, as entidades descritas foram cliente, pizza e pedido.

As informações do cliente pertinentes ao programa da pizzeria são:

- **nome do cliente:** para identificar quem receberá a pizza no endereço de entrega. Além disso, o atendimento telefônico é mais agradável se o cliente for chamado pelo nome;
- **telefone:** número que aparece no identificador de chamadas da pizzeria e que será digitado na tela do programa para identificar o endereço;

- **endereço:** para o entregador ir até o cliente. O endereço é composto por várias partes: logradouro, número, complemento, bairro, cidade, estado e CEP;
- **ponto de referência:** não é obrigatório, mas quando informado auxilia o entregador a localizar o endereço.

As informações do pedido pertinentes ao programa da pizzaria são – número, data, hora e valor:

- **número:** para identificar o pedido;
- **data:** para saber quando o pedido foi feito;
- **hora:** para determinar quanto tempo passou do momento do pedido até a entrega;
- **valor:** para determinar o quanto será cobrado do cliente.

Uma forma gráfica de representar as entidades Cliente e Pedido é por meio do Diagrama Entidade-Relacionamento (DER). Nesse diagrama, a entidade é representada por um retângulo, e os atributos, por círculos, como demonstrado na Figura 3.

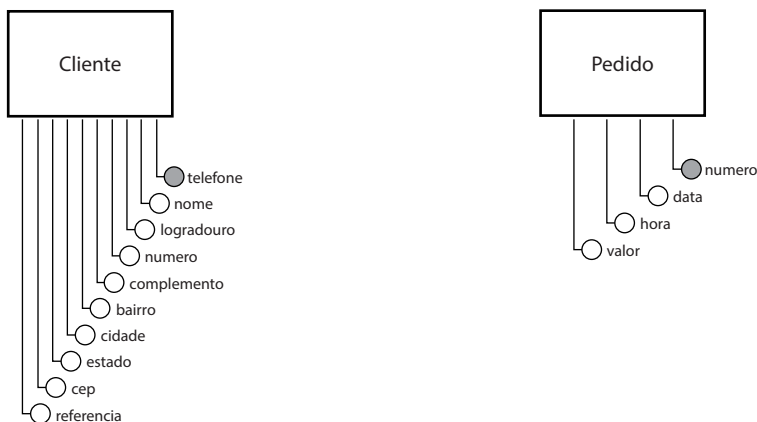


Figura 3 – Entidades com seus atributos.

O atributo identificador é representado pelo círculo preenchido e os demais atributos, por círculos vazios.

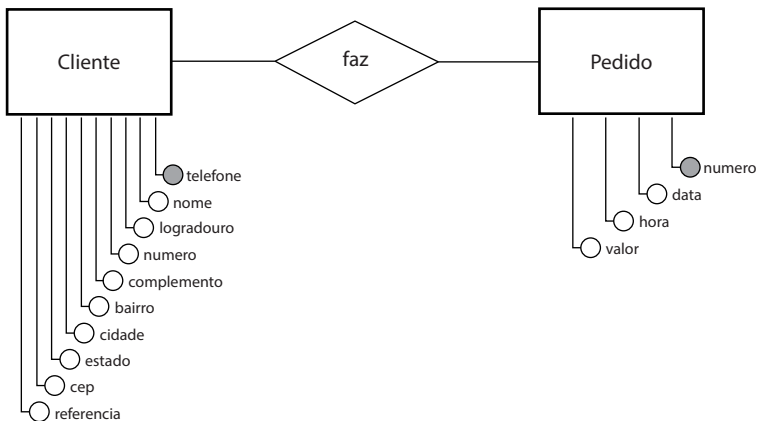
Como o nome do diagrama demonstra, falta representar o relacionamento que existe entre as duas entidades.

Estes são alguns exemplos de relacionamento:

- clientes **fazem** pedidos;
- autores **escrevem** livros;
- alunos **cursam** disciplinas.

Sempre há duas entidades e um verbo que estabelece a relação entre elas.

Para representar essa relação utiliza-se um losango, conforme se observa na Figura 4, a seguir.



Ivan Sardella

Figura 4 – Relacionamento entre entidades.

Outra informação relevante que deve ser registrada no diagrama é a cardinalidade. Ela determina as quantidades mínima e máxima de vezes que a relação entre as entidades ocorre.

Um cliente pode fazer quantos pedidos?

Quando o cliente faz seu primeiro pedido, ainda não tem cadastro. Nesse momento, o atendente solicitará e registrará essas informações. Conclui-se que, para ser cliente da pizzeria, é necessário realizar pelo menos um pedido.

O cliente poderá, a partir do primeiro pedido, realizar quantos pedidos quiser. Tecnicamente, o cliente pode fazer “n” pedidos.

Em resumo, o cliente faz pelo menos um pedido ou quantos pedidos quiser. Isso é representado como **(1, n)**, o que indica que a cardinalidade mínima é 1 e a máxima é “n”.

E como fica a cardinalidade do pedido em relação ao cliente?

O pedido número 10, por exemplo, só pode ser feito por um único cliente. Se esse pedido pertencesse a dois ou mais clientes, em qual endereço ele seria entregue?

A cardinalidade do pedido em relação ao cliente é **(1, 1)**, o que indica que todo pedido deve ter um, e somente um, cliente.

Essas anotações aparecem no DER, conforme mostrado na Figura 5.

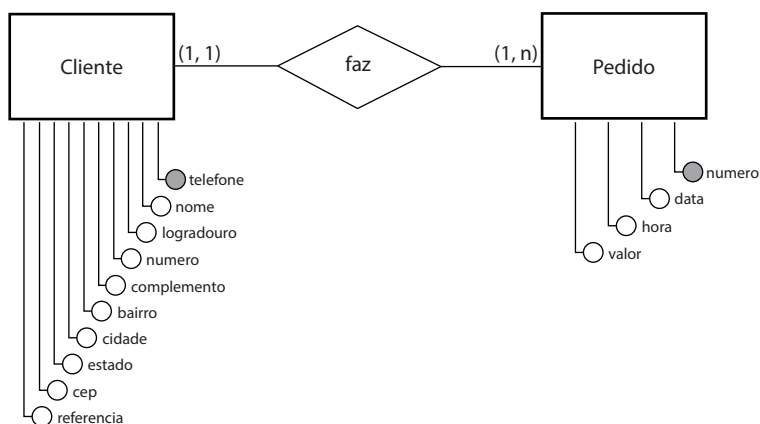


Figura 5 – Exemplo de cardinalidade.

Até agora o pedido não contém o detalhamento dos produtos: quais pizzas foram escolhidas pelo cliente que resultam no total do pedido?

A pizza tem um código para sua identificação no cardápio, um nome, uma descrição e o valor.

A pizza portuguesa é uma das mais solicitadas. Todos os dias são feitos “n” pedidos que contêm essa pizza.

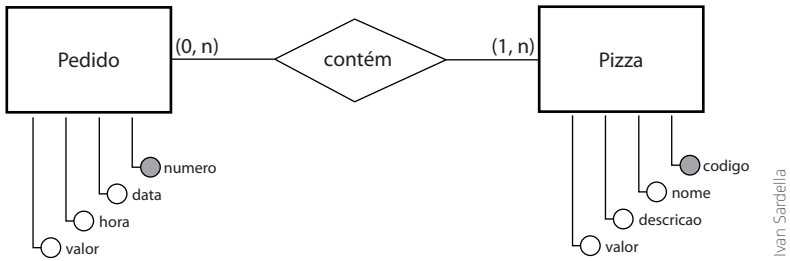


Figura 6 – Relacionamento e cardinalidades entre pedido e pizza.

O relacionamento mostrado na Figura 6 determina que o pedido deve conter uma ou mais pizzas. Isso é correto porque fazer um pedido e não informar qual a pizza equivale a não fazer o pedido. Por isso, a cardinalidade de pedido com pizza é $(1, n)$.

Alguns sabores de pizza, como portuguesa, são clássicos e vendem muito bem. Portanto, essa pizza pode “estar contida” em “n” pedidos. Existe a possibilidade de a pizzeria ter criado um sabor que nunca tenha sido vendido, por isso a pizza pode “estar contida” em nenhum pedido. Então, a cardinalidade de pizza com pedido é $(0, n)$.

A modelagem de dados deve responder a perguntas do tipo “quais pedidos de ontem tinham pizza portuguesa?” ou “quantas pizzas portuguesas foram vendidas na semana passada?”.

Recuperar qual pedido contém a pizza portuguesa significa determinar quando (data, hora) e por qual cliente ela foi solicitada, mas o modo como a modelagem foi feita não permite obter esse tipo de informação. É necessário criar uma entidade “pizzas de pedido”, que guarde os sabores e as quantidades de pizza que o cliente pediu.

Esse tipo de entidade é chamado de entidade associativa e, como o nome sugere, ela associa duas outras entidades para poder identificar os detalhes dessa relação.

Uma entidade associativa básica contém os atributos identificadores das duas entidades que estão sendo associadas, necessários para cumprir seu papel associativo. Se for necessário, é possível ter outros atributos na entidade associativa.

No caso da entidade associativa “pizzas do pedido” serão necessários o número do pedido (atributo identificador do pedido), o código da pizza (atributo identificador da pizza), a quantidade solicitada dessa pizza e o valor unitário dela.

É importante lembrar que a entidade associativa situa temporalmente uma informação. Por isso, indica-se o valor unitário da pizza no detalhe do pedido. Caso ocorra alteração de preço, os pedidos antigos precisam manter a informação do que aconteceu antes da alteração de preço.

A Figura 7 representa a entidade associativa. Deve-se observar que os dois símbolos, da entidade e da associação, se fundem em um único, mas cada um possui seu nome.

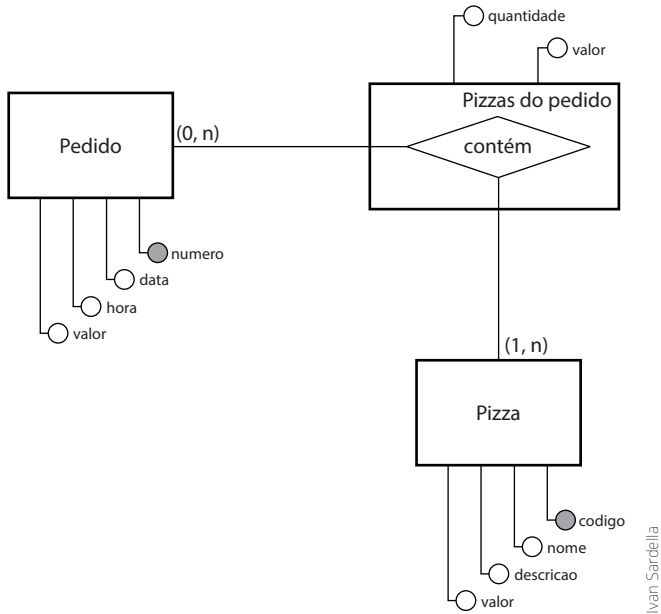


Figura 7 – Exemplo de entidade associativa.

Outra maneira de representar a cardinalidade entre as entidades é anotar o valor das cardinalidades máximas no relacionamento, conforme mostrado na Figura 8.

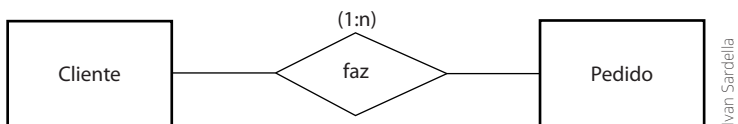


Figura 8 – Anotação da cardinalidade no relacionamento.

Entre cliente e pedido as cardinalidades são (1, 1) e (1, n). Toma-se a cardinalidade máxima de cliente, que é 1, e a cardinalidade máxima de pedido, que é “n”, e o grau de relacionamento é 1:n (lê-se: um para n), conforme indicado na Figura 8.

A Figura 9, a seguir, contém o DER da modelagem conceitual completa para as regras de negócio fornecidas pelo proprietário da pizzaria:

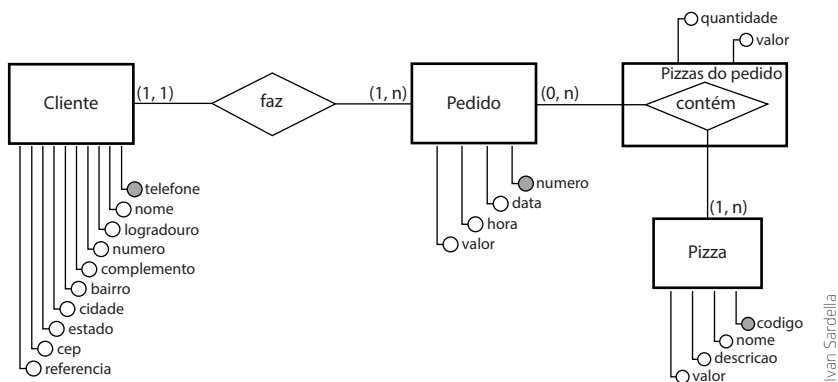


Figura 9 – Diagrama Entidade-Relacionamento (DER) da pizzaria.

Modelo lógico

O modelo lógico é utilizado para representar as entidades na forma de tabelas e os atributos como colunas dessas tabelas.

Os atributos identificadores tornam-se chaves primárias, e as referências de uma entidade em outra passam a ser tratadas como chave estrangeira.

Os atributos devem ter seu tipo definido de acordo com seu significado:

- **NUMERIC(n, d)** para colunas que armazenam números. O “n” indica a quantidade de dígitos da parte inteira do número e o “d”, a quantidade de dígitos decimais.
- **VARCHAR(n)** para colunas que armazenam textos com até “n” caracteres.
- **CHAR(n)** para colunas que armazenam textos com “n” caracteres.
- **BOOLEAN** para colunas que armazenam valores lógicos.
- **DATETIME** para colunas que armazenam datas e horas.

O modelo lógico para cliente deve ser criado tendo por base algumas considerações:

- telefone é um texto e seu tamanho pode variar de 1 até 10 caracteres;
- telefone é a chave primária, pois era o atributo identificador;
- nome, logradouro, complemento, bairro, cidade e referência são textos com tamanho que variam de 1 até 30 caracteres;
- número é um campo numérico inteiro com 5 dígitos;
- cep é um campo numérico inteiro com 8 dígitos;
- estado é um texto com 2 caracteres.

Cliente	
	telefone: VARCHAR(10)
	nome: VARCHAR(30)
	logradouro: VARCHAR(30)
	numero: NUMERIC(5,0)
	complemento: VARCHAR(30)
	bairro: VARCHAR(30)
	cidade: VARCHAR(30)
	estado: CHAR(2)
	cep: NUMERIC(8,0)
	referencia: VARCHAR(30)

Acervo do autor

Figura 10 – Modelo lógico da entidade Cliente.

Na Figura 10, que representa a entidade Cliente no modelo lógico, observa-se que:

- a primeira linha identifica a entidade de acordo com o nome definido no modelo conceitual;
- as demais linhas contêm as definições dos tipos de dado de acordo com a regra de negócio descrita anteriormente, observando que a chave primária é identificada pelo desenho de uma chave dourada.

Para fazer o modelo lógico da entidade Pedido, deve-se detalhar o tipo de cada coluna de acordo com o atributo que ela representa; para isso, é importante lembrar que o tipo DATETIME armazena data e hora, o que faz dos atributos data e hora uma única coluna.

Entre as entidades Cliente e Pedido, existe a relação “faz”, que deve ser representada no modelo lógico, pois é necessário indicar qual cliente fez o pedido. Para identificar um cliente, utiliza-se a coluna “telefone” que é a chave primária.

A ideia da chave primária é que, ao informar seu valor, o cliente será identificado sem a possibilidade de erro. Por essa razão, não é necessário anotar todos os dados do cliente no pedido, apenas seu telefone.

Na Figura 11, o telefone do cliente deve ser o mesmo anotado na Figura 10, mas não pode ser uma chave primária, visto que pela regra no negócio quem assume esse papel é a coluna “número”.

Por ser uma chave primária em outra tabela e representar uma relação, o telefone do cliente, na Figura 11, é chamado de chave estrangeira e será representado pelo desenho de uma chave cinza.



Figura 11 – Exemplo de modelo físico com chave estrangeira.

A Figura 12 demonstra o modelo físico com todas as entidades.

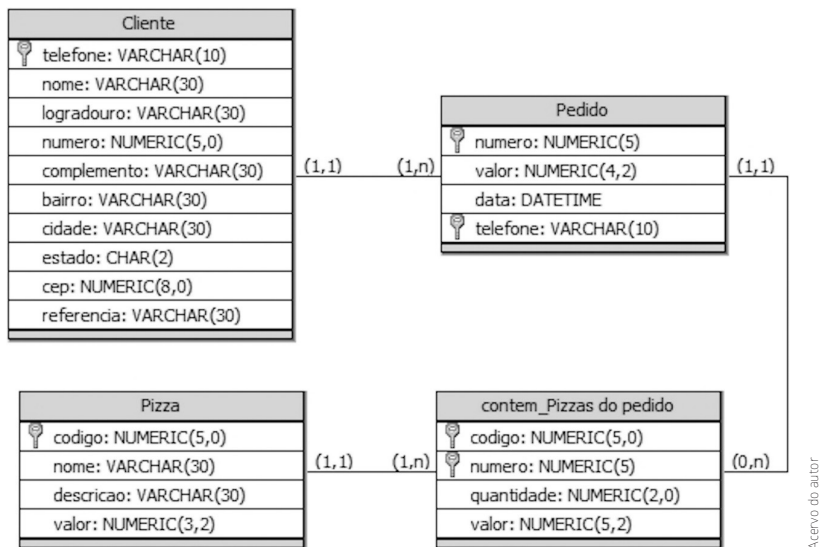


Figura 12 – Modelo lógico da pizzaria.

Modelo físico

O modelo físico equivale aos comandos necessários para a criação das tabelas no banco de dados. Posteriormente, será estudado o comando CREATE TABLE e elaborado o modelo físico da pizzaria.

RECAPITULANDO

Este capítulo apresentou os conceitos básicos sobre banco de dados, Modelo Entidade-Relacionamento, como definir entidades e seus atributos e como representar um modelo lógico e físico de um banco de dados.

Exercícios

Para os cenários a seguir, criar modelos conceitual e lógico.

1. Uma empresa deseja informatizar suas operações. Inicialmente, quer manter um cadastro de todos os seus clientes, a partir do armazenamento de informações como RG, nome, telefone e endereço.

Deseja também manter um cadastro que contenha dados sobre os produtos que vende, como nome do produto, preço e quantidade em estoque. Quando um cliente faz uma compra, informações sobre quem fez a compra, a data da compra, o valor total e os produtos comprados são armazenados.

2. Uma empresa comercializa produtos de limpeza e deseja controlar os produtos que vende, seus clientes e os pedidos por meio da utilização de um programa de computador.

Cada produto tem código, nome, categoria (por exemplo: sabão em barra, sabão em pó, sabão líquido etc.) e seu preço. A categoria é uma classificação criada pela própria empresa.

A firma possui informações sobre todos os seus clientes. Cada cliente é identificado por código, nome, endereço, telefone, *status* (“cinco estrelas”, “três estrelas”, “uma estrela”) e o seu limite de crédito.

Também são armazenadas as informações dos pedidos feitos pelos clientes. Cada pedido possui um número e a data de elaboração. Todos os pedidos devem ter um ou mais produtos associados a ele, com a quantidade solicitada pelo cliente.

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

2. Linguagem de Definição de Dados (DDL)

Comando SHOW

Comando CREATE DATABASE

Comando DROP DATABASE

Comando USE

Comando CREATE TABLE

Comando DROP TABLE

Comando ALTER TABLE

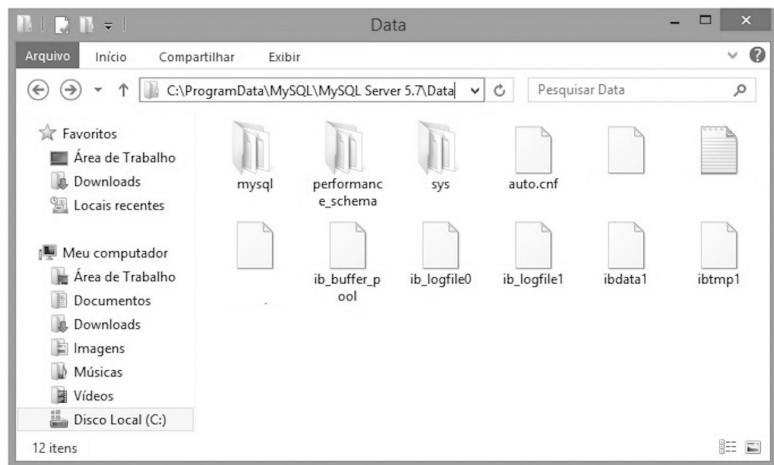
A SQL (*Structured Query Language*, ou Linguagem de Consulta Estruturada) é a linguagem utilizada para executar comandos em um banco de dados relacional, como o MySQL.

O grupo de comandos conhecido como DDL (*Data Definition Language*, ou Linguagem de Definição de Dados) é utilizado para a criação e manutenção de esquemas, tabelas e índices, dentre outros.

No processo de modelagem do banco de dados, são criados os modelos conceitual e lógico e, por fim, o modelo físico que é escrito com os comandos DDL do banco de dados.

É importante observar que normalmente se fala “utilizo o banco de dados MySQL”, mas isso é incorreto. O MySQL é um Sistema Gerenciador de Banco de Dados (SGBD), um programa que cuida da organização física e lógica dos bancos de dados.

Um banco de dados equivale a uma pasta (ou diretório) na qual são gravados os arquivos manipulados pelo SGBD.



Acervo do autor

Figura 1 – Estrutura de diretório do MySQL.

Na Figura 1, observa-se a pasta “Data” na instalação padrão do MySQL no Windows. As pastas “mysql”, “performance_schema” e “sys” são bancos de dados gerenciados pelo MySQL.

Este capítulo aborda:

- como fazer manutenção em um banco de dados no MySQL;
- os principais tipos de dados do MySQL;
- como fazer manutenção em tabelas de um banco de dados.

Comando SHOW

Antes de iniciar a criação e manutenção de um banco de dados, é importante conhecer o comando que exibe informações sobre vários elementos estruturais.

A sintaxe do comando é:

```
SHOW <elemento>;
```

Por exemplo:

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql          |
| performance_schema |
| sys            |
+-----+
4 rows in set (0.00 sec)
```

- “mysql>” é o *prompt* de comandos do MySQL. Ele indica que o MySQL está pronto para receber um comando;
- “show databases;” é o comando que exibir os bancos existentes no MySQL.

Observar que todo comando no console deve ser concluído com ponto e vírgula.

A resposta ao comando é a relação de bancos e a linha com dados estatísticos.

Para conhecer todos os elementos estruturais exibidos pelo comando “show” utiliza-se a ajuda do MySQL pelo comando “help show;”.

Comando CREATE DATABASE

O comando CREATE DATABASE (ou CREATE SCHEMA) é utilizado para criar um banco de dados no MySQL. É no banco que serão criadas as tabelas e manipulados os seus dados.

A sintaxe do comando é:

```
CREATE DATABASE <nome do banco>;
```

Por exemplo:

```
mysql>create database exemplo;  
Query OK, 1 row affected (0.00 sec)
```

A primeira linha solicita a criação do banco de dados exemplo e a segunda linha é a resposta que o MySQL retorna após a execução do comando. Nesse caso, a resposta indica que o comando (*query*) foi executado com sucesso e consumiu-se 0.00 segundo para a conclusão do comando.

Utilizar o comando SHOW DATABASES para confirmar a criação do banco:

```
mysql> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| exemplo            |  
| mysql              |  
| performance_schema |  
| sys                |  
+-----+  
5 rows in set (0.00 sec)
```

Caso o nome utilizado para a criação do banco já exista, a resposta indicará a ocorrência de um erro com sua descrição detalhada:

```
ERROR 1007 (HY000): Can't create database 'exemplo';  
database exists
```

A tradução desse erro é “Não é possível criar o banco de dados ‘exemplo’; banco de dados existe”.

Na criação de um *script* de banco, em que vários comandos serão executados em sequência, pode-se evitar esse tipo de erro a partir da utilização de um comando de decisão associado ao comando de criação do banco.

A sintaxe do comando é:

```
CREATE DATABASE IF NOT EXISTS <nome do banco>;
```

Por exemplo:

```
mysql> create database if not exists exemplo;  
Query OK, 1 row affected, 1 warning (0.00 sec)
```

A primeira linha solicita a criação do banco de dados exemplo caso este ainda não exista e a segunda linha é a resposta que o MySQL retorna após a execução do comando. Nesse caso, a resposta indica que o comando (*query*) foi executado com sucesso, existe um alerta ou advertência após a sua execução e consumiu-se 0.00 segundo para a conclusão do comando.

Comando DROP DATABASE

O comando DROP DATABASE (ou DROP SCHEMA) é utilizado para remover um banco de dados no MySQL.

A sintaxe do comando é:

```
DROP DATABASE <nome do banco>;
```

Por exemplo:

```
mysql> drop database exemplo;
Query OK, 0 row affected (0.00 sec)
```

A primeira linha solicita a remoção do banco de dados exemplo e a segunda linha é a resposta que o MySQL retorna após a execução do comando. Nesse caso, a resposta indica que o comando (*query*) foi executado com sucesso e consumiu-se 0.00 segundo para a conclusão do comando.

Utilizar o comando SHOW DATABASES para confirmar a remoção do banco:

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.00 sec)
```

Caso o nome de banco que se deseja remover não exista, a resposta indicará a ocorrência de um erro com sua descrição detalhada:

```
ERROR 1008 (HY000): Can't drop database 'exemplo';
database doesn't exist
```

A tradução desse erro é “Não é possível remover o banco de dados ‘exemplo’; banco de dados não existe”.

Na criação de um *script* de banco, vários comandos serão executados em sequência, pode-se evitar esse tipo de erro a partir da utilização de um comando de decisão associado ao comando de criação do banco.

A sintaxe do comando é:

```
DROP DATABASE IF EXISTS <nome do banco>;
```

Por exemplo:

```
mysql> drop database if exists exemplo;  
Query OK, 0 rows affected, 1 warning (0.00 sec)
```

A primeira linha solicita a criação do banco de dados exemplo caso ainda não exista e a segunda linha é a resposta que o MySQL retorna após a execução do comando. Nesse caso, a resposta indica que o comando (*query*) foi executado com sucesso, existe um alerta ou advertência após sua execução e consumiu-se 0.00 segundo para a conclusão do comando.

Comando USE

O MySQL organiza os bancos de dados como pastas (ou diretórios) no sistema operacional. Para acessar os arquivos de uma pasta, é necessário “entrar” nessa pasta. Da mesma maneira, para ter acesso às tabelas de um banco de dados, é necessário “entrar” no banco. O termo correto é “usar” o banco e, para isso, utiliza-se o comando USE.

Para os exemplos utilizados a partir daqui, é necessário criar o banco de dados pizzaria.

A sintaxe do comando é:

```
USE <nome do banco>;
```

Por exemplo:

```
mysql> use pizzaria;  
Database changed
```

A primeira linha solicita a mudança para o banco de dados pizzaria e a segunda linha é a resposta que o MySQL retorna com a indicação de que o comando foi executado com sucesso e agora MySQL está “dentro” do banco pizzaria.

Caso o nome de banco que se deseja utilizar não exista, a resposta indicará a ocorrência de um erro com sua descrição detalhada.

```
ERROR 1049 (42000): Unknown database 'pizzarias'
```

A tradução desse erro é “Banco de dados desconhecido ‘pizzarias’”.

Comando CREATE TABLE

O comando CREATE TABLE é utilizado para criar uma tabela no banco de dados em uso no MySQL. A sintaxe básica do comando é:

```
CREATE TABLE [IF NOT EXISTS] <nome da tabela> (  
    <nome da coluna 1> <tipo> <definição>,  
    <nome da coluna 2> <tipo> <definição>,  
    <nome da coluna n> <tipo> <definição>,  
);
```

Por exemplo:

```
mysql> use pizzaria;
mysql> create table pizza (
  -> codigo integer,
  -> nome varchar(30),
  -> valor decimal(5,2)
  -> );
Query OK, 0 rows affected (0.22 sec)
```

Após a execução desse comando, foi criada a tabela PIZZA no banco de dados pizzaria. Observar que, durante a digitação de comandos no console do MySQL, se for digitado ENTER para abrir uma nova linha, é colocado o símbolo “->” para indicar que se trata de continuação da linha anterior. Enquanto não for digitado o ponto e vírgula, o comando não será executado.

A tabela foi criada com a coluna código do tipo inteiro com 11 dígitos, a coluna nome do tipo caractere com 30 dígitos e a coluna valor do tipo real com três dígitos antes da vírgula e dois dígitos após a vírgula.

Utilizar o comando SHOW TABLES para confirmar a criação da tabela:

```
mysql> show tables;
+-----+
| Tables_in_pizzaria |
+-----+
| pizza               |
+-----+
```

Ao tentar criar uma tabela com um nome já existente, será devolvida uma resposta com erro:

```
ERROR 1050 (42S01): Table 'pizza' already exists
```

A tradução desse erro é “Tabela ‘pizza’ já existe”. Para evitá-lo, usar a cláusula IF NOT EXISTS, o que indica que a tabela só será criada caso o nome não exista.

Tipos de dados

No exemplo anterior, utilizou-se alguns tipos de dados para criar a tabela PIZZA. A linguagem SQL define alguns tipos de dados, mas cada SGBD pode definir outros tipos além desses.

A Tabela 1 apresenta os tipos de dados da linguagem SQL utilizados no MySQL.

Tabela 1 – Tipos de dados da linguagem SQL utilizados no MySQL

Tipo de dado	Descrição
CHAR(n)	Caractere de comprimento fixo.
VARCHAR(n)	Caractere de comprimento variável.
DECIMAL(n)	Valor numérico com n dígitos e sinal.
NUMERIC(n)	Idêntico ao DECIMAL(n).
INTEGER ou INT	Equivale a DECIMAL(11,0).
DECIMAL(n,d)	Valor numérico com n dígitos e sinal antes do ponto decimal, e d dígitos após o ponto decimal.
NUMERIC(n,d)	Idêntico ao DECIMAL(n,d)
DATE	Data no formato ‘AAAA-MM-DD’, como em um calendário.
TIME	Hora no formato ‘HH:MM:SS’, como em um relógio.
DATETIME	Data e hora no formato ‘AAAA-MM-DD HH:MM:SS’, a data como em um calendário e a hora como em um relógio.
TIMESTAMP	Data e hora no formato ‘AAAA-MM-DD HH:MM:SS’, considerando-se o fuso horário e tendo como valor mínimo ‘1970-01-01 00:00:01’ UTC e valor máximo ‘2038-01-19 03:14:07’ UTC.

Valor padrão

Caso a coluna tenha um valor padrão, aquele que será gravado na coluna se um valor não for informado, utiliza-se a cláusula `DEFAULT` para determinar esse valor. O exemplo a seguir cria a coluna `ingredientes` e determina que o valor “Massa, molho, queijo” será gravado se a coluna não for informada no comando de inserção:

```
...
-> ingredientes varchar(30) default "Massa, molho,
queijo",
...
```

Not null

Uma das restrições é a obrigatoriedade da coluna, ou seja, durante a inserção de dados ela deve ter valor informado ou não. Se não é uma coluna obrigatória, utiliza-se a cláusula `NULL`, que é o padrão, e caso seja uma coluna obrigatória, utiliza-se a cláusula `NOT NULL`.

O exemplo a seguir cria a coluna `ingredientes` e `extras` como não obrigatória, sendo que a primeira usa o `NULL` como padrão e a segunda emprega o `NULL` de forma explícita:

```
...
-> ingredientes varchar(30),
-> extras varchar(30) null,
...
```

O próximo exemplo cria a coluna `ingredientes`, mas agora como obrigatória:

```
...
-> ingredientes varchar(30) not null,
...
```

Unique

Outra restrição determina se o valor informado para aquela coluna deve ou não ser único. Se a coluna permitir valores repetidos, não se indica nada na sua criação. Se a coluna não permitir valores repetidos, utilizar a cláusula `UNIQUE` ou `UNIQUE KEY` para determinar essa restrição.

O exemplo a seguir cria a coluna `nome` e determina que os valores inseridos não podem se repetir:

```
...  
-> nome varchar(30) not null unique,  
...
```

Outra sintaxe do comando `UNIQUE` é:

```
...  
-> unique (nome);  
...
```

Nesta sintaxe, informar o nome de mais de uma coluna que se deseja restringir; é importante separar os nomes por vírgula.

Chave primária

Se uma coluna é obrigatória (`NOT NULL`) e não pode ter valores repetidos (`UNIQUE`), pode ser utilizada como identificador único da tabela. Existe uma cláusula chamada `PRIMARY KEY` (chave primária) que equivale a essa combinação de restrições. A coluna anotada como `PRIMARY KEY` equivale ao atributo identificador de uma entidade, conforme utilizado no modelo lógico.

O exemplo a seguir anota a coluna `código` como chave primária, ou seja, é obrigatória (*not null*) e não pode conter valores repetidos (*unique*). Essa sintaxe permite anotar apenas uma coluna.

```
...  
-> codigo integer primary key,  
...
```

Outra sintaxe do comando PRIMARY KEY é:

```
...  
-> primary key (codigo),  
...
```

Nessa sintaxe, é possível informar o nome de mais de uma coluna, os nomes devidamente separados por vírgula, de modo a formar uma chave primária composta.

Auto_increment

Se a coluna será anotada como chave primária e é de tipo numérico inteiro (INTEGER), pode-se solicitar ao MySQL para preenchê-la automaticamente com a cláusula AUTO_INCREMENT. O banco será responsável por gerar o número que preencherá essa coluna e a incrementará um a um.

O exemplo a seguir anota a coluna código como chave primária e capaz de produzir seus valores automaticamente. Essa sintaxe permite anotar apenas uma coluna:

```
...  
-> codigo integer primary key auto_increment,  
...
```

Outra sintaxe do comando é:

```
...  
-> codigo integer auto_increment,  
...  
-> primary key (codigo),  
...
```

Nessa sintaxe, é possível informar o nome de mais de uma coluna, cujos nomes devem estar devidamente separados por vírgula, de maneira a formar uma chave primária composta.

Chave estrangeira

Quando a informação salva em uma coluna se refere a uma chave primária de outra tabela, essa coluna é chamada de chave estrangeira (*foreign key*). Deve-se informar na criação da tabela quais campos terão esse comportamento e quais tabelas e campos serão referenciados.

Por exemplo, a tabela PEDIDO da pizzaria tem uma coluna para identificar o cliente. Essa coluna guarda o valor da coluna telefone da tabela CLIENTE.

Na tabela PEDIDO, deve-se registrar um código que aponte para uma linha de outra tabela. Isso é o relacionamento, discutido anteriormente.

A sintaxe do comando para declaração de chave estrangeira é:

```
CONSTRAINT <nome da chave>  
    FOREIGN KEY (<coluna de chave estrangeira>)  
    REFERENCES <tabela> (<coluna de chave primária>)
```

A cláusula CONSTRAINT (não obrigatória) permite dar um nome à chave estrangeira. A cláusula FOREIGN KEY determina qual coluna da tabela atual será chave estrangeira e a cláusula REFERENCES assinala a tabela e a consulta que serão pesquisadas como chave primária.

O exemplo a seguir cria uma coluna para armazenar o código do cliente na tabela de pedidos e anota essa coluna como chave estrangeira, que aponta para o código do cliente na tabela CLIENTE:

```
...
-> telefone varchar(10) unique not null,
-> constraint "fk_cliente"
->   foreign key (telefone)
->   references cliente(telefone),
...
```

Desse modo, só poderão ser inseridos pedidos de clientes que estiverem cadastrados.

Comando DROP TABLE

O comando DROP TABLE é utilizado para remover uma tabela no banco de dados em uso no MySQL.

A sintaxe básica do comando é:

```
DROP TABLE [IF EXISTS] <nome da tabela>;
```

Por exemplo:

```
mysql> drop table pizza;
Query OK, 0 rows affected (0.14 sec)
```

Caso o nome da tabela que se deseja remover não exista no banco de dados, será devolvida resposta com erro:

```
ERROR 1051 (42S02): Unknown table 'pizzaria.pizza'
```

A tradução desse erro é “Tabela ‘pizzaria.pizza’ não existe”. Para evitá-lo, utilizar a cláusula IF EXISTS; só se removerá a tabela caso o nome não exista.

Comando ALTER TABLE

O comando ALTER TABLE é utilizado para alterar a estrutura de uma tabela existente no banco de dados em uso no MySQL. Essa alteração de estrutura pode ser a adição de uma nova coluna, a remoção de uma coluna ou a alteração do tipo de uma coluna.

A sintaxe básica do comando para adicionar uma coluna é:

```
ALTER TABLE <nome da tabela>  
  ADD <nome> <tipo> <definição>;
```

Se o nome da coluna que se deseja adicionar já existir, será devolvida resposta com erro.

Além de uma coluna, é possível adicionar PRIMARY KEY e FOREIGN KEY como definições da nova coluna ou em uma linha de comando separada.

A sintaxe básica do comando para alterar uma coluna é:

```
ALTER TABLE <nome da tabela>  
  MODIFY <nome> <tipo> <definição>;
```

O comando MODIFY é utilizado para alterar o tipo e a definição de colunas já existentes.

A sintaxe básica do comando para renomear uma coluna é:

```
ALTER TABLE <nome da tabela>  
  CHANGE <nome> <novo nome> <tipo>;
```

Se o nome da coluna não existir ou se o novo nome já existir, será devolvida resposta com erro.

A sintaxe básica do comando para remover uma coluna é:

```
ALTER TABLE <nome da tabela> DROP <nome da coluna>;
```

Se o nome da coluna que se deseja excluir não existir, será devolvida resposta com erro.

RECAPITULANDO

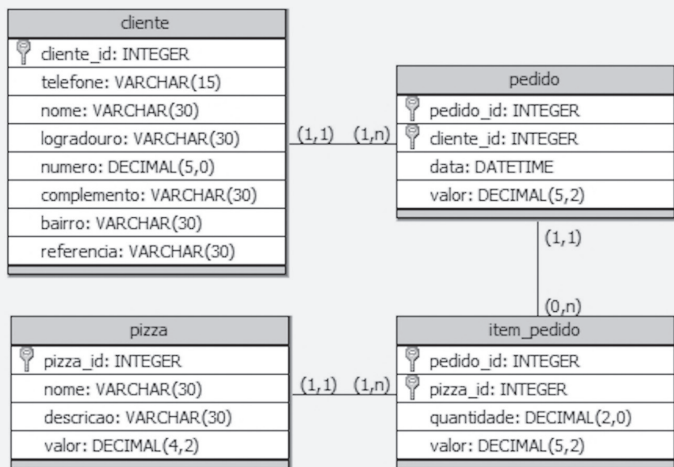
O conjunto de comandos conhecidos como DDL, responsável pela criação e manutenção de banco de dados e tabelas, foi estudado neste capítulo.

Esses comandos são utilizados para a criação do modelo físico, último processo na modelagem de dados.

A linguagem SQL permite diversas variações em sua sintaxe, e aqui foram utilizadas apenas algumas dessas variações. Após a conclusão dos exercícios deste capítulo, é recomendável consultar a documentação oficial do MySQL na internet.

Exercícios

1. Continuar a criação do modelo físico em um banco de dados chamado “pizzaria” para as tabelas “pedido”, “pizza” e “item_pedido” do modelo lógico de pizzaria apresentado na figura a seguir.



Arquivo do autor

Modelo lógico de pizzaria.

```
-- Remover as tabelas, caso existam
DROP TABLE IF EXISTS item_pedido;
DROP TABLE IF EXISTS pedido;
DROP TABLE IF EXISTS cliente;
DROP TABLE IF EXISTS pizza;
-- Tabela Cliente
CREATE TABLE Cliente (
  cliente_id DECIMAL(5,0) AUTO_INCREMENT,
  telefone VARCHAR(10),
  nome VARCHAR(30),
  logradouro VARCHAR(30),
  numero DECIMAL(5,0),
  complemento VARCHAR(30),
  bairro VARCHAR(30),
  referencia VARCHAR(30),
  PRIMARY KEY (cliente_id)
);
```

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

3. Linguagem de Manipulação de Dados (DML)

Comando INSERT

Comando SELECT

Comando DELETE

Comando UPDATE

O grupo de comandos da linguagem SQL conhecido como DML (*Data Manipulation Language*, ou Linguagem de Manipulação de Dados) é utilizado para a criação e manutenção de dados nas tabelas.

Quando um usuário preenche um formulário em um programa instalado em seu PC, celular, tablet ou na internet, ao clicar no botão SALVAR, as informações serão persistidas em um banco de dados.

Posteriormente, esse usuário pode alterar ou excluir informações. Essas ações serão executadas, novamente, em um banco de dados.

O usuário, por meio do programa, pode executar comandos para inclusão, exclusão, alteração e consulta em um banco de dados. Essas ações são conhecidas como CRUD (*Create, Read, Update e Delete*, em inglês) e são comuns em programação.

Este capítulo aborda como:

- inserir dados em uma tabela;
- executar consultas em uma tabela;
- alterar dados em uma tabela;
- excluir dados de uma tabela.

Comando INSERT

Para inserir novos dados em um banco de dados, utiliza-se o comando INSERT. Sua sintaxe é:

```
insert into <tabela>
    (coluna 1, coluna 2, coluna n)
values
    (<valor coluna 1>, <valor coluna 2>, <valor coluna n>);
```

Se o nome da tabela ou das colunas não existir ou o valor informado não for compatível com a definição da coluna, será devolvida resposta com erro.

Em outra sintaxe possível não são informados os nomes das colunas e nos parênteses do VALUES são informados todos os valores das colunas, na sequência em que foram criadas:

```
insert into <tabela>
values
    (<valor coluna 1>, <valor coluna 2>, <valor coluna n>);
Por exemplo:
insert into pizza
    (codigo, nome, valor)
values
    (1, "Portuguesa", 15.00);
```

O comando anterior vai inserir uma nova linha na tabela PIZZA, que atribuirá o valor à coluna código, "4 queijos" à coluna nome e R\$ 25,00 à coluna valor.

```
insert into pizza
values
    (2, "4 Queijos", 25.00);
```

O comando anterior vai inserir uma nova linha na tabela PIZZA da mesma maneira que o comando do exemplo anterior. O valor 2 será atribuído à coluna código, pois foi a primeira coluna criada na definição da tabela, e assim sucessivamente para os demais comandos.

Na definição da tabela PIZZA, a coluna código foi anotada como chave primária (PRIMARY KEY), o que significa ser obrigatória (NOT NULL) e não permitir valores repetidos (UNIQUE). Se o comando INSERT da pizza portuguesa for executado novamente, será devolvida a resposta com o seguinte erro:

```
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
```

A tradução deste erro é “Entrada duplicada ‘1’ para a chave primária”.

Além de chave primária, a coluna código foi anotada como AUTO_INCREMENT, o que significa ser capaz de produzir valores automaticamente para essa coluna, de modo a evitar o problema de chave duplicada. O exemplo a seguir mostra como utilizar o comando INSERT para essa situação:

```
insert into pizza  
  (nome, valor)  
values  
  ("Portuguesa", 15.00);
```

O nome das colunas pode ser informado em qualquer sequência desde que os valores sejam informados na mesma sequência. Vale observar também:

- Colunas anotadas como NOT NULL são obrigatórias e deverão ser informadas no INSERT.
- Colunas anotadas como FOREIGN KEY deverão ter valor correspondente na tabela que referenciam.
- Colunas anotadas como DEFAULT serão preenchidas com o valor anotado na criação da tabela se não tiverem valor informado.

Executar os comandos a seguir para inserir algumas linhas na tabela de pizzas e clientes criadas no exercício do capítulo anterior. Estas informações serão necessárias para os próximos tópicos.

```
INSERT INTO pizza (nome, valor) VALUES ('Portuguesa', 15);
INSERT INTO pizza (nome, valor) VALUES ('Provolone', 17);
INSERT INTO pizza (nome, valor) VALUES ('4 Queijos', 20);
INSERT INTO pizza (nome, valor) VALUES ('Calabresa', 17);
INSERT INTO pizza (nome) VALUES ('Escarola');
INSERT INTO `cliente` VALUES (1, '(11) 1111-1111',
'Alexandre Santos',
'Rua das Palmeiras', 111, NULL, 'Bela Vista', 'Em frente
a escola');
```

Comando SELECT

Para executar consultas em um banco de dados, utiliza-se o comando SELECT. Sua sintaxe é:

```
select [distinct] <colunas>
  from <tabelas>
  [where condição]
  [group by <coluna>]
  [having <condição>]
  [order by <coluna>];
```

A consulta mais simples que se pode realizar é solicitar todas as colunas de todas as linhas de uma tabela. O exemplo a seguir mostra isso com a tabela PIZZA, que utiliza o asterisco para representar “todas as colunas”:


```
select * from pizza;
```

pizza_id	nome	descrição	valor
1	Portuguesa	NULL	15.00
2	Provolone	NULL	17.00
3	4 Queijos	NULL	20.00
4	Calabresa	NULL	17.00
5	Escarola	NULL	NULL

É possível indicar o nome de cada coluna em vez de usar o asterisco e obter o mesmo resultado:

```
select pizza_id, nome, valor from pizza;
```

pizza_id	nome	valor
1	Portuguesa	15.00
2	Provolone	17.00
3	4 Queijos	20.00
4	Calabresa	17.00
5	Escarola	NULL

A sequência das colunas pode ser alterada como for necessário. Por exemplo, para exibir o valor, o código e o nome da pizza:

```
select valor, pizza_id, nome from pizza;
```

Também é possível selecionar algumas colunas em vez de todas. Por exemplo, para exibir o nome e o valor das pizzas:

```
select nome, valor from pizza;
```

O nome da coluna exibido na resposta do comando pode ser alterado com a cláusula AS:

```
select pizza_id as codigo, nome, valor as R$ from pizza;
```

codigo	nome	R\$

Operações aritméticas

O comando SELECT é extremamente flexível e pode ser utilizado não apenas para consultar dados em tabelas, mas também com os operadores aritméticos para executar cálculos.

```
select 10+3, 10-3, 10/3, 10*3;
```

10+3	10-3	10/3	10*3
13	7	3.3333	30

Nesse exemplo, o nome das colunas não ficou significativo. Ao utilizar a cláusula AS, tem-se:

```
select 10+3 as SOMA, 10-3 as SUBTRACAO, 10/3 as
MULTIPLICACAO;
```

SOMA	SUBTRACAO	DIVISAO	MULTIPLICACAO
13	7	3.3333	30

As operações aritméticas podem ser executadas com as colunas de uma tabela. Por exemplo, para saber qual o valor das pizzas se for aplicado um acréscimo de 10%, tem-se:

```
select pizza_id, nome, valor, valor*1.1 as "novo valor"
from pizza;
```

pizza_id	nome	valor	novo valor
1	Portuguesa	15.00	16.500
2	Provolone	17.00	18.700
3	4 Queijos	20.00	22.000
4	Calabresa	17.00	18.700
5	Escarola	NULL	NULL

Cláusula WHERE

A cláusula WHERE é utilizada para determinar quais linhas da tabela serão listadas, como se fosse um filtro. Para executar esse filtro, são utilizados os operadores relacionais, que retornam “verdadeiro” ou “falso” e determinam se a linha será ou não exibida.

Existe a possibilidade de não haver linhas que se apliquem a esses filtros e isso não é considerado erro, apenas não há retorno de dados.

Os operadores relacionais do MySQL estão indicados no Quadro 1.

Quadro 1 – Operadores relacionais

Operador	Descrição
>	Maior que
>=	Maior ou igual a
<	Menor que
<=	Menor ou igual a
=	Igual a
<> ou !=	Diferente de
<=>	Igual para NULL

Os exemplos a seguir demonstram o uso da cláusula WHERE com os operadores relacionais. Será descrito o que se deseja fazer, o comando e a resposta do banco de dados.

- Selecionar todas as pizzas com valor superior a R\$ 17,00:

```
select * from pizza where valor > 17;
```

codigo	nome	descricao	valor
3	4 Queijos	NULL	20.00

- Selecionar todas as pizzas com valor igual ou superior a R\$ 17,00:

```
select * from pizza where valor >= 17;
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
3	4 Queijos	NULL	20.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valor inferior a R\$ 17,00:

```
select * from pizza where valor < 17;
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00

- Selecionar todas as pizzas com valor igual ou inferior a R\$ 17,00:

```
select * from pizza where valor <= 17;
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00
2	Provolone	NULL	17.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valor igual a R\$ 17,00:

```
select * from pizza where valor = 17;
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valor diferente de R\$ 17,00:

```
select * from pizza where valor != 17;
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00
3	4 Queijos	NULL	20.00

- Selecionar todas as pizzas com a coluna valor nula:

```
select * from pizza where valor <=> null;
```

codigo	nome	descricao	valor
5	Escarola	NULL	NULL

Os operadores lógicos podem ser combinados em expressões lógicas, em que cada teste retornará verdadeiro ou falso e, devidamente combinados na expressão, resultarão em um único “verdadeiro” ou único “falso”, o que determinará se a linha será exibida na consulta.

Para criar uma expressão lógica, utilizam-se os operadores lógicos E (AND ou &&), OU (OR ou ||) e NÃO (NOT ou !).

- Selecionar todas as pizzas com valores entre R\$ 15,00 e R\$ 20,00:

```
select * from pizza where valor > 15 and valor < 20;
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valores menores ou maiores do que R\$ 17,00:

```
select * from pizza where valor < 17 OR valor > 17;
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00
3	4 Queijos	NULL	20.00

- Selecionar todas as pizzas com valores não menores ou maiores do que R\$ 17,00:

```
select * from pizza where not (valor < 17 OR valor > 17);
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
4	Calabresa	NULL	17.00

Existem outros operadores que atuam como facilitadores no processo de filtragem de dados. Utiliza-se BETWEEN para especificar intervalos de dados, IN para verificar se o dado faz parte de um

conjunto específico de valores, IS para tratar dados nulos e LIKE para aplicar filtros em colunas de caracteres.

Nos exemplos a seguir, será exibido o comando sem o facilitador e com o facilitador.

- Selecionar todas as pizzas com valores entre R\$ 17,00 e R\$ 20,00:

```
select * from pizza where valor >= 17 and valor <= 20;
select * from pizza where valor between 17 and 20;
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
3	4 Queijos	NULL	20.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valores fora da faixa entre R\$ 17,00 e R\$ 20,00:

```
select * from pizza where valor < 17 or valor > 20;
select * from pizza where not (valor >= 17 and valor <=
20);
select * from pizza where valor not between 17 and 20;
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
3	4 Queijos	NULL	20.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valores iguais a R\$ 15,00 e R\$ 20,00:

```
select * from pizza where valor = 15 or valor = 20;
select * from pizza where valor in (15, 20);
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00
3	4 Queijos	NULL	20.00

- Selecionar todas as pizzas com valores diferentes de R\$ 15,00 e R\$ 20,00:

```
select * from pizza where valor != 15 and valor != 20;
select * from pizza where valor not in (15, 20);
```

codigo	nome	descricao	valor
2	Provolone	NULL	17.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valores nulos:

```
select * from pizza where valor <=> null;
select * from pizza where valor is null;
```

codigo	nome	descricao	valor
5	Escarola	NULL	NULL

- Selecionar todas as pizzas com valores diferentes de nulo:

```
select * from pizza where not valor <=> null;
select * from pizza where valor is not null;
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00
2	Provolone	NULL	17.00
3	4 Queijos	NULL	20.00
4	Calabresa	NULL	17.00

Apesar de o operador LIKE ser utilizado amplamente para o tratamento de colunas do tipo CHAR e VARCHAR, também pode ser usado para colunas com tipos numéricos.

Quando precedido do operador NOT, faz a seleção contrária à do solicitado pelo LIKE.

LIKE faz uso de alguns símbolos para determinar o que será ou não listado. Esses caracteres especiais são chamados de coringas (*wildcards*, em inglês). O Quadro 2 mostra quais são eles e sua função.

Quadro 2 – Caracteres coringas

Coringa	Função
%	Indica nenhum, um ou vários caracteres
–	Indica um, e somente um, caractere

Nos exemplos a seguir, será exibido o comando com o operador LIKE e a resposta devolvida pelo banco.

- Selecionar todas as pizzas com nomes que comecem com a letra E:

```
select * from pizza where nome like "E%";
```

+	-----	+	-----	+	-----	+	-----	+
	codigo		nome		descricao		valor	
+	-----	+	-----	+	-----	+	-----	+
	5		Escarola		NULL		NULL	
+	-----	+	-----	+	-----	+	-----	+

- Selecionar todas as pizzas com nomes que terminem com a letra A:

```
select * from pizza where nome like "%A";
```

+	-----	+	-----	+	-----	+	-----	+
	codigo		nome		descricao		valor	
+	-----	+	-----	+	-----	+	-----	+
	1		Portuguesa		NULL		15.00	
	4		Calabresa		NULL		17.00	
	5		Escarola		NULL		NULL	
+	-----	+	-----	+	-----	+	-----	+

- Selecionar todas as pizzas com nomes que tenham as letras “ESA” em qualquer posição:

```
select * from pizza where nome like "%RE%";
```

codigo	nome	descricao	valor
1	Portuguesa	NULL	15.00
4	Calabresa	NULL	17.00

- Selecionar todas as pizzas com valores cujo dígito da unidade seja 0. O *underscore* representará qualquer valor no dígito da dezena, o que determinará a obrigatoriedade de existir esse dígito. Se houver uma pizza com valor menor que R\$ 1,00, esta não deve ser listada por não ter um caractere na posição da dezena:

```
select * from pizza where valor like "_0%";
```

codigo	nome	descricao	valor
3	4 Queijos	NULL	20.00

Apelido de tabela

Existem situações em que duas ou mais tabelas estão relacionadas no mesmo comando SELECT com a possibilidade de haver colunas com o mesmo nome em cada uma das tabelas.

Por exemplo, as tabelas PIZZAS e CLIENTES têm uma coluna chamada “nome”. Listar o nome de todos os clientes e o nome de todas as pizzas no mesmo comando produzirá erro, pois são duas colunas com o mesmo nome pertencentes a tabelas diferentes:

```
select nome, nome from cliente, pizza;
ERROR 1052 (23000): Column 'nome' in field list is
ambiguous
```

A tradução do erro é “Coluna ‘nome’ na lista de campos é ambígua”, ou seja, o MySQL reconhece que as duas tabelas têm essa coluna, mas não pode decidir qual tabela fornecerá os dados para as colunas.

Para corrigir esse erro, utiliza-se o nome da tabela antes da coluna, para identificar formalmente a origem daquela informação:

```
select cliente.nome, pizza.nome from cliente, pizza;
```

nome	nome
Alexandre Santos	Portuguesa
Alexandre Santos	Provolone
Alexandre Santos	4 Queijos
Alexandre Santos	Calabresa
Alexandre Santos	Escarola

Outra maneira de escrever esse comando é a criação de apelido às tabelas, por meio da utilização da palavra AS ou com base na inclusão do apelido da tabela após seu nome:

```
select c.nome, p.nome from cliente as c, pizza p;
```

nome	nome
Alexandre Santos	Portuguesa
...	
Alexandre Santos	Escarola

Como a resposta está com a palavra “nome” em duas colunas, é interessante dar apelidos para as colunas, o que ajuda a evitar interpretações errôneas de seus conteúdos:

```
select c.nome Cliente, p.nome Pizza from cliente as c,
pizza p;
```

Cliente	Pizza
Alexandre Santos	Portuguesa
...	
Alexandre Santos	Escarola

Observar que a consulta de cliente e pizza na forma registrada neste tópico tem caráter ilustrativo para explicar o comando. Posteriormente, serão estudadas as formas de unir tabelas em um SELECT, o que dá significado às informações obtidas.

Cláusula ORDER BY

As linhas da resposta de um comando SELECT (após filtragem pela cláusula WHERE e agrupamentos, que serão estudados posteriormente) podem ser ordenadas por uma ou mais colunas, de forma crescente (ascendente) ou decrescente (descendente). A cláusula de ordenação deverá ser a última de um comando SELECT.

A sintaxe do comando é:

```
...
order by <coluna 1> [asc/desc], <coluna n> [asc/desc];
```

Caso não seja informada a ordem desejada, a coluna será ordenada de forma crescente.

Os exemplos a seguir ilustram o uso da cláusula ORDER BY.

- Listar todas as colunas da tabela PIZZA em ordem alfabética:

```
select * from pizza order by nome asc;
select * from pizza order by nome;
```

codigo	nome	descricao	valor
3	4 Queijos	NULL	20.00
4	Calabresa	NULL	17.00
5	Escarola	NULL	22.00
1	Portuguesa	NULL	15.00
2	Provolone	NULL	17.00

- Listar todas as colunas da tabela PIZZA em ordem decrescente de preço:

```
select * from pizza order by valor desc;
```

codigo	nome	descricao	valor
5	Escarola	NULL	22.00
3	4 Queijos	NULL	20.00
2	Provolone	NULL	17.00
4	Calabresa	NULL	17.00
1	Portuguesa	NULL	15.00

- Listar o nome e o valor das pizzas em ordem decrescente de valor e crescente de nome. A primeira coluna a ser ordenada será a do bairro, em ordem alfabética inversa. Nos dados já ordenados, a coluna nome será organizada em ordem alfabética, em respeito à ordenação já promovida na primeira parte. Observar as pizzas que custam R\$ 17,00 com destaque para:

```
select nome, bairro from cliente order by valor desc,  
nome;
```

nome	valor
Escarola	22.00
4 Queijos	20.00
Calabresa	17.00
Provolone	17.00
Portuguesa	15.00

Comando DELETE

Para remover dados de um banco de dados, utiliza-se o comando DELETE. Sua sintaxe é:

```
delete from <tabela> where <condição>;
```

Se o nome da tabela ou das colunas da condição não existir ou o valor informado não for compatível com a definição da coluna, será devolvida resposta com erro.

Todas as linhas que atenderem à condição serão removidas da tabela. Por exemplo:

- Excluir o cliente com código 100:

```
delete from cliente where telefone = "1234-5678";
```

- Excluir os pedidos anteriores a 25/12/2016:

```
delete from pedido where data_hora < "2016-12-25";
```

Todas as possibilidades de condição estudadas no comando SELECT aplicam-se ao comando DELETE e caso se execute um comando DELETE sem condição, **todas as linhas da tabela serão removidas**, portanto, esse comando deve ser utilizado com bastante atenção.

Comando UPDATE

Para alterar dados existentes em um banco de dados, utiliza-se o comando UPDATE. Sua sintaxe é:

```
update <tabela>  
  set <coluna 1>=<valor 1>, <coluna n>=<valor n>  
  where <condição>;
```

Se o nome da tabela ou das colunas não existir ou o valor informado não for compatível com a definição da coluna, será devolvida resposta com erro.

Na tabela PIZZAS, a linha da escarola foi cadastrada sem preço. Para alterar esse valor, utiliza-se:

```
update pizza set valor=22.00 where codigo = 5;
```

No exemplo, o valor será alterado para R\$ 22,00 na linha em que o código seja igual a 5.

Todas as possibilidades de condição estudadas no comando SELECT aplicam-se ao comando UPDATE e, caso se execute um comando UPDATE sem condição, **todas as linhas da tabela serão alteradas**, portanto, esse comando deve ser utilizado com bastante atenção.

RECAPITULANDO

Este capítulo tratou da manipulação de informações de um banco de dados, que ocorre por meio dos comandos DML, que devem seguir regras ou restrições definidas na criação das tabelas.

O comando `INSERT`, em todas suas variações, é utilizado para inserir novas linhas nas tabelas; o comando `DELETE` remove linhas já existentes da tabela; e o comando `UPDATE` é utilizado para alterar o conteúdo de linhas inseridas anteriormente. Nos comandos `DELETE` e `UPDATE`, é imperativo o uso da cláusula `WHERE` para evitar que todas as linhas da tabela sejam excluídas ou alteradas, a menos que esta seja a ação que se deseja.

Para consultar o conteúdo de uma tabela, o comando `SELECT` apresenta diversas cláusulas que permitem determinar a ordem das linhas de resposta, bem como a aplicação de filtros para determinar o que deve ser descartado na consulta.

Estes são os comandos utilizados em uma aplicação desenvolvida em linguagem de programação como Java ou C# e, por isso, é de grande importância que os programadores dominem suas sintaxes.

Exercícios

1. Executar as inserções das linhas nas devidas tabelas considerando os relacionamentos entre elas.

Se eventualmente houver erro durante a inserção das linhas, utilizar o comando `UPDATE` para corrigir o que estiver errado.

Clientes

Nome	Telefone	Endereço	Número	Bairro	Referência
Bruna Dantas	(22) 2222-2222	Rua das Rosas	222	Cantareira	
Carlos Vieira	(33) 3333-3333	Rua das Avencas	333	Bela Vista	
Julia Silva	(44) 4444-4444	Rua dos Cravos	444	Cantareira	Esquina do mercado
Mariana Silva	(55) 5555-5555	Rua das Acácias	555	Bela Vista	
Laura Madureira	(66) 6666-6666	Rua das Gardênias	666	Cantareira	

Pedidos

Número	Cliente	Data e hora	Valor
1	1	15/12/2016 20:30	R\$ 32.00
2	2	15/12/2016 20:38	R\$ 40.00
3	3	15/12/2016 20:59	R\$ 22.00
4	1	17/12/2016 22:00	R\$ 42.00
5	2	18/12/2016 19:00	R\$ 45.00
6	3	18/12/2016 21:12	R\$ 44.00
7	4	19/12/2016 22:22	R\$ 72.00
8	6	19/12/2016 22:26	R\$ 34.00

Itens de pedido

Número	Pizza	Quantidade	Valor total
1	1	1	15.00
1	4	1	17.00
2	3	2	40.00
3	5	1	22.00
4	3	1	20.00
4	5	1	22.00
5	1	3	45.00
6	5	2	44.00
7	1	2	30.00
7	3	1	20.00
7	5	1	22.00
8	4	2	34.00

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

4. Funções

Funções matemáticas

Funções de *string*

Funções de data e hora

Funções de agregação

Subconsulta

Como a maioria dos aplicativos que manipulam dados, o MySQL possui uma biblioteca de funções para facilitar tarefas comuns ou a execução de cálculos.

As funções estão agrupadas de acordo com sua finalidade, por exemplo, funções matemáticas, de data e hora e de agregação. Esse último grupo atua sobre diversas linhas do banco de dados, o que produz um único resultado.

As funções podem ser utilizadas para consultas, o que é bastante comum, mas também nas inclusões e alterações das colunas ou na cláusula WHERE, de modo muito versátil.

Posteriormente, será tratada a criação de funções pelo programador, o que aumenta a flexibilidade do banco de dados.

Este capítulo aborda:

- as funções matemáticas do MySQL;
- as funções de caracteres do MySQL;
- as funções de data e hora do MySQL;
- as funções de agregação do MySQL;
- como executar uma subconsulta.

Funções matemáticas

Utilizadas para realizar cálculos, as funções matemáticas são aplicadas a uma constante ou valor de coluna ou retornam um valor. O Quadro 1 apresenta algumas das funções matemáticas do MySQL.

Quadro 1 – Funções matemáticas

Nome	Descrição
ABS(n)	Valor absoluto de “n”
CEILING(n)	Arredonda “n” para o próximo valor inteiro
CONV(n, base, base)	Converte um valor de uma base para outra base
DIV	Divisão entre inteiros
FLOOR(n)	Arredonda “n”, de modo a eliminar sua parte decimal
MOD ou %	Resto da divisão
PI()	Valor de PI
POW(b, e)	Eleva a base “a” ao expoente “e”
RAND(n)	Gera um valor aleatório que usa “n” como semente. Se “n” não é informado, gera um número aleatório com semente própria
ROUND(n, c)	Arredonda o valor “n” e mantém “c” casas após o ponto. Se “c” não é informado, arredonda “n” para um valor inteiro
SIGN(n)	Retorna 1 se “n” for positivo e -1 se “n” for negativo
SQRT(n)	Retorna à raiz quadrada de “n”
TRUNCATE(n, d)	Devolve o valor “n” com “d” casas decimais

As funções matemáticas podem ser utilizadas com constantes. Por exemplo:

```
select sqrt(144) as "raiz";
+-----+
| raiz |
+-----+
|    12 |
+-----+
```

Também são utilizadas funções com as colunas de uma tabela. Por exemplo:

```
select quantidade, valor, truncate((valor / quantidade),2)
as valor_unitario from item_pedido;
```

quantidade	valor	valor_unitario
1	15.00	15.00
1	17.00	17.00
2	40.00	20.00
1	22.00	22.00
1	20.00	20.00
1	22.00	22.00
3	45.00	15.00
2	44.00	22.00
2	30.00	15.00
1	20.00	20.00
1	22.00	22.00
2	34.00	17.00

Para conhecer todas as funções matemáticas, deve-se consultar a documentação oficial ou digitar “*help numeric functions*” no console do MySQL.

Funções de *string*

Utilizadas para a manipulação de texto, as funções de *string* são aplicadas a uma constante ou coluna e retornam uma *string*. O Quadro 2 apresenta algumas das funções de *string* do MySQL.

Quadro 2 – Funções de *string*

Nome	Descrição
CONCAT(str1, str2, ...)	Concatena diversas <i>strings</i>
LEFT(str, n)	“n” caracteres da esquerda de “str”
LOWER(str)	“str” com todos os caracteres minúsculos
LENGTH(str)	Quantidade de caracteres de “str”
RIGHT(str)	“n” caracteres da direita de “str”
SUBSTRING(str, p, q)	“q” caracteres a partir da posição “p” de “str”
UPPER(str)	“str” com todos os caracteres maiúsculos

As funções de *string* podem ser utilizadas com constantes. Por exemplo:

```
select substr("Mariana Silva", 4, 5) as "Parte do nome";
+-----+
| Parte do nome |
+-----+
| e Sil        |
+-----+
```

Também são utilizadas funções com colunas de uma tabela. Por exemplo:

```
select upper(substr(nome, 4, 5)) as "Parte do nome" from
cliente;
+-----+
| Parte do nome |
+-----+
| XANDR        |
| NA DA        |
| NO VI        |
| LIA S        |
| E SIL        |
| RA MA        |
+-----+
```

Para conhecer todas as funções de *string*, deve-se consultar a documentação oficial ou digitar “*help string functions*” no console do MySQL.

Funções de data e hora

Utilizadas para a manipulação de dados relacionados à marcação do tempo, as funções de data e hora são aplicadas a uma constante ou coluna e retornam uma *string*. O Quadro 3 apresenta algumas das funções de data e hora do MySQL.

Quadro 3 – Funções de data e hora

Nome	Descrição
CURDATE()	Data atual
CURTIME()	Hora atual
DATEDIFF(d1, d2)	Diferença, em dias, entre “d1” e “d2”
DAY(d)	Dia da data “d”
HOURL(h)	Hora (0 a 23) da hora “h”
MINUTE(h)	Minuto (0 a 59) da hora “h”
MONTH(d)	Mês da data “d”
MONTHNAME(d)	Nome do mês da data “d”
NOW()	Data e hora atuais
SECOND(h)	Segundo (0 a 59) da hora “h”
YEAR(d)	Ano da data “d”

Algumas das funções de data retornam textos. Se o MySQL estiver com a configuração padrão, os textos estarão em inglês. Para alterar o idioma do banco para português, deve-se configurar a variável `LC_TIME_NAMES` com `pt_BR`. O comando para essa alteração é:

```
set lc_time_names = 'pt_BR';
```

As funções de data e hora podem ser utilizadas com constantes. Por exemplo:

```
select datediff('2016-12-31', '2016-12-25') as "Dias";
```

Dias
6

Também são utilizadas funções com colunas de uma tabela. Por exemplo:

```
select datediff('2016-12-31', data) as dias from pedido;
```

dias
16
16
16
14
13
13
12
12

Para conhecer todas as funções de data e hora, deve-se consultar a documentação oficial ou digitar *“help date and time functions”* no console do MySQL.

Funções de agregação

Conhecidas como funções estatísticas, as funções de agregação obtêm informação sobre conjuntos de linhas especificados na cláusula WHERE. O Quadro 4 apresenta algumas das funções de agregação do MySQL.

Quadro 4 – Funções de agregação

Nome	Descrição
AVG(coluna)	Média dos valores da coluna
COUNT	Número de linhas
MAX(coluna)	Maior valor da coluna
MIN(coluna)	Menor valor da coluna
SUM(coluna)	Soma dos valores da coluna

As funções de agregação são utilizadas com colunas de uma tabela. Por exemplo:

- Qual é a média de preço das pizzas?

```
select avg(valor) from pizza;
+-----+
| avg(valor) |
+-----+
| 18.200000 |
+-----+
```

- Quantos clientes estão cadastrados na pizzaria? A função “count” pode ser utilizada de modo a informar como parâmetro um asterisco ou o nome de uma coluna da tabela. No pri-

meio caso, serão contadas as linhas em que a coluna informada possui conteúdo diferente de nulo. No segundo caso, valores nulos não são computados.

```
select count(*) as quantidade from pizza;
```

```
+-----+
| quantidade |
+-----+
|          5 |
+-----+
```

- Qual é a pizza mais cara do cardápio da pizzeria?

```
select max(valor) as "Mais cara" from pizza;
```

```
+-----+
| Mais cara |
+-----+
|      22.00 |
+-----+
```

- Qual é a pizza mais barata do cardápio da pizzeria?

```
select min(valor) as "Mais barata" from pizza;
```

```
+-----+
| Mais barata |
+-----+
|      15.00 |
+-----+
```

- Qual é o valor total do pedido número 7?

```
select sum(valor) as "Total do pedido 7"
from item_pedido
where pedido_id=7;
```

```
+-----+
| Total do pedido 7 |
+-----+
|           72.00 |
+-----+
```

Subconsulta

Como o nome sugere, subconsulta é um SELECT executado dentro de outro SELECT.

Supondo que o proprietário da pizzeria necessite da relação de pizzas com valor abaixo do valor médio das pizzas do cardápio:

- Primeiro, pesquisar o valor médio das pizzas:

```
select avg(valor) from pizza;
+-----+
| avg(valor) |
+-----+
| 18.200000 |
+-----+
```

- Com base no valor médio das pizzas do cardápio, é possível pesquisar as pizzas com valor abaixo dele:

```
select * from pizza where valor < 18.2;
+-----+-----+-----+-----+
| pizza_id | nome      | descricao | valor |
+-----+-----+-----+-----+
| 1 | Portuguesa | NULL      | 15.00 |
| 2 | Provolone  | NULL      | 17.00 |
| 3 | Calabresa  | NULL      | 17.00 |
+-----+-----+-----+-----+
```

A primeira consulta pode ser adicionada à segunda, de modo a aumentar a flexibilidade do comando SELECT:

```
select * from pizza where valor < (select avg(valor) from
pizza);
```

Observar que a subconsulta deve estar entre parênteses e retornar um único valor a partir da utilização de uma função de agregação.

RECAPITULANDO

O MySQL oferece uma variedade de funções que auxiliam o programador a obter informações de forma prática e segura. Elas estão organizadas de acordo com sua utilidade e apresentam sintaxe bastante simples.

Se não fosse pelas funções, para obter dados estatísticos o programador deveria consultar uma tabela do banco de dados, mover o resultado para uma planilha eletrônica e nela executar cálculos ou contagens.

É importante que o programador, independentemente da linguagem de programação que utiliza, esteja familiarizado com as funções e delegue para o banco as tarefas de obter as informações de modo que o programa realize apenas os processamentos que não podem ser realizados pelo banco.

Exercícios

1. Escrever os comandos para:
 - a) Listar o pedido mais antigo.
 - b) Listar o pedido mais recente.
 - c) Listar o pedido com o maior valor.

- d) Listar o pedido com o menor valor.
- e) Listar, em relação à ordem alfabética, o nome do primeiro cliente da pizzeria.
- f) Listar, em relação à ordem alfabética, o nome do último cliente da pizzeria.
- g) Listar o nome dos clientes em letras maiúsculas.
- h) Listar a soma dos pedidos realizados no dia 18/12/2016.
- i) Listar a diferença, em dias, entre os pedidos mais atual e mais antigo, com base na subconsulta.

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

5. Consultas com informações agrupadas

Cláusula GROUP BY

Cláusula HAVING

Cláusula DISTINCT

Operador UNION

Como visto no Capítulo 4, com o uso das funções de agrupamento, foi possível obter um resultado a partir de várias linhas de uma tabela. Esse tipo de informação é bastante útil, mas é comum a necessidade de dados agrupados de acordo com algum critério.

Saber o valor das vendas de uma empresa é uma informação valiosa, mas saber como foram as vendas mês a mês pode auxiliar na tomada de decisão de maneira mais precisa.

O MySQL possui algumas cláusulas no comando SELECT que permitem esse tipo de agrupamento.

Este capítulo aborda como:

- totalizar resultados de grupos de dados;
- aplicar filtros para grupos de dados;
- unir resultados de consultas.

Cláusula GROUP BY

As funções de agregação utilizadas anteriormente fornecem dados importantes no comando SELECT. É possível aumentar a flexibilidade desses comandos com a cláusula GROUP BY que, como o nome sugere, executa uma função de agregação, que agrupa os dados de acordo com outra coluna.

Por exemplo, na tabela ITEM_PEDIDO é possível obter o valor total de cada pedido ao utilizar a função SUM(VALOR_TOTAL) para agrupar a soma pelo número do pedido. Quando houver troca no número do pedido, o MySQL começa um novo como SUM, mesmo que as linhas não estejam organizadas por essa coluna.

Para ilustrar o funcionamento dessa cláusula, deve-se observar o comando e o resultado a seguir:

```
select pedido_id as "Número do Pedido", sum(valor) as  
      "Valor Pedido" from item_pedido;
```

Número do Pedido	Valor Pedido
1	331.00

O MySQL utilizou a primeira linha da tabela para obter o valor da coluna “Número do Pedido” e executou a soma de todas as linhas da coluna “Valor”. Essa informação está errada, pois o valor

R\$ 331,00 equivale à soma de todos os pedidos e não somente do pedido número 1.

O comando e a resposta a seguir demonstram como utilizar a cláusula GROUP BY para agrupar a soma do valor de acordo com o número do pedido:

```
select pedido_id as "Número do Pedido", sum(valor) as  
"Valor Pedido" from item_pedido group by pedido_id;
```

Número do Pedido	Valor Pedido
1	32.00
2	40.00
3	22.00
4	42.00
5	45.00
6	44.00
7	72.00
8	34.00

O MySQL ordenará a tabela de acordo com o número do pedido, que foi a coluna informada, e não será agregada e executada a soma, de modo a totalizar cada vez que o número do pedido for alterado.

Pode-se utilizar quantos comandos de agregação forem necessários em um comando SELECT com GROUP BY, porém, deve-se informar a mesma coluna referência (na declaração de colunas a serem listadas no SELECT e que será utilizada como referencial para o GROUP BY) para manter a integridade dos dados.

Cláusula HAVING

Na criação de resultados agrupados é possível aplicar filtros para determinar quais linhas deverão participar do agrupamento. Isso é feito por meio da cláusula HAVING, complementar à cláusula GROUP BY.

Por exemplo, a partir da tabela ITEM_PEDIDO, para listar o valor total de cada pedido somente para pedidos com valor superior a R\$ 40,00, deve-se utilizar o comando executado anteriormente acrescido da cláusula HAVING, como demonstrado a seguir:

```
select pedido_id as "Número do Pedido",  
       sum(valor) as "Valor Pedido" from item_pedido  
       from item_pedido  
       group by pedido_id  
       having sum(valor) > 40;
```

+-----+-----+	
Número do Pedido	Valor Pedido
+-----+-----+	
4	42.00
5	45.00
6	44.00
7	72.00
+-----+-----+	

O MySQL organizará a tabela em ordem crescente da coluna NÚMERO e executará a soma da coluna VALOR_TOTAL. Se o valor da soma se adequar ao filtro da cláusula HAVING, será devolvida a linha de resposta. Caso contrário, a linha será ignorada e não constará na resposta.

Cláusula DISTINCT

A cláusula DISTINCT do comando SELECT elimina respostas duplicadas de uma pesquisa. A tabela ITEM_PEDIDO contém o número dos pedidos realizados pela pizzeria. Para obter uma listagem com esses números, pode-se fazer conforme comando a seguir:

```
select pedido_id as "Número" from item_pedido;
```

Número
1
1
2
3
4
4
5
6
7
7
7
8

O inconveniente dessa resposta é a repetição dos números dos pedidos. O ideal seria listá-los uma única vez. Para isso, utiliza-se a cláusula DISTINCT antes do nome da coluna de que se deseja eliminar as repetições. O comando anterior com essa cláusula seria:

```
select distinct pedido_id as "Número" from item_pedido;
```

Número
1
2
3
4
5
6
7
8

Operador UNION

O operador UNION concatena duas respostas de comando SELECT. É importante que os dois SELECTs possuam o mesmo número de colunas para que a união ocorra.

Os nomes das colunas na resposta serão os informados no primeiro SELECT.

Por exemplo, para criar uma listagem com o código e o nome das pizzas e dos clientes a partir da aplicação do operador UNION, utiliza-se:

```
select pizza_id, nome from pizza
UNION
select cliente_id, nome from cliente;
```

pizza_id		nome
1		Portuguesa
2		Provolone
3	4	Queijos
4		Calabresa
5		Escarola
1		Alexandre Santos
2		Bruna Dantas
3		Carlos Vieira
4		Julia Silva
5		Mariana Silva
6		Laura Madureira

Os comandos SELECT poderão ter restrições aplicadas com a cláusula WHERE. Por exemplo, para listar somente as pizzas com código ímpar e os clientes com código par, utiliza-se:

```
select pizza_id, nome from pizza where pizza_id%2 != 0
union
select cliente_id, nome from cliente where cliente_id%2 =
0;
```

pizza_id	nome
1	Portuguesa
3	4 Queijos
5	Escarola
2	Bruna Dantas
4	Julia Silva
6	Laura Madureira

As consultas unidas poderão ter apenas uma cláusula ORDER BY, pois essa cláusula é aplicada após a montagem completa da resposta do comando SELECT.

```
select pizza_id, nome from pizza where pizza_id%2 != 0
union
select cliente_id, nome from cliente where cliente_id%2 = 0
order by nome;
```

pizza_id	nome
3	4 Queijos
2	Bruna Dantas
5	Escarola
4	Giulia Silva
6	Laura Madureira
1	Portuguesa

O comando UNION não exibe linhas duplicadas, portanto, se houver duas linhas com informações idênticas, somente uma será exibida.

RECAPITULANDO

Executar consultas que determinem filtros e ordenação para a resposta pode não ser suficiente para atender às regras de negócio que se deseja implementar. Às vezes, pode ser necessário agrupar dados de modo que sejam gerados relatórios sintéticos.

A visão sintética, ou resumida, dos dados auxilia as empresas na tomada de decisão. Por isso, este capítulo de agrupamento de dados é tão significativo.

Unir resultados distintos de duas consultas também auxilia na formação de uma resposta de pesquisa mais complexa e completa, o que permite a extração de informação complexa da base de dados de forma simples, em uma única linha de comando.

Exercícios

1. O proprietário da pizzeria necessita dos seguintes relatórios:
 - a) A quantidade de vendas de cada pizza em ordem decrescente de quantidades.
 - b) O código do cliente e, em ordem decrescente de valor, o total de seus pedidos desde que é cliente da pizzeria.

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

6. Junção de tabelas

Relação entre tabelas

Cláusula INNER JOIN

Cláusula LEFT JOIN

Cláusula RIGHT JOIN

Desde a modelagem dos dados, há a preocupação em determinar qual a relação existente entre duas entidades.

Até o momento, essas relações estão sendo consideradas para garantir, por exemplo, que um cliente deverá ser cadastrado para então realizar um pedido e que uma pizza não possa ser excluída caso esteja presente em algum pedido.

Neste capítulo, o relacionamento será estendido para as consultas, com o objetivo de permitir a obtenção de informações mais completas e realizar consultas para representar a regra de negócio.

O proprietário da pizzeria pode solicitar uma relação das pizzas que estão no cadastro, mas nunca foram vendidas, ou dos clientes que realizaram grandes volumes de compra em um determinado período. Essas consultas exigem o relacionamento de duas ou mais tabelas.

Este capítulo aborda como:

- relacionar duas ou mais tabelas;
- executar consultas do tipo INNER JOIN;
- executar consultas do tipo OUTER JOIN.

Relação entre tabelas

Ao listar a tabela ITEM_PEDIDO tem-se:

```
select * from item_pedido;
```

pedido_id	pizza_id	quantidade	valor
1	1	1	15.00
1	4	1	17.00
	...		
8	4	2	34.00

Os desenvolvedores compreendem o significado dos números na coluna “pizza_id” bem como sabem que, por meio do “pedido_id”, podem identificar o código do cliente e obter seus dados.

Mas como estabelecer essas relações?

Quando se executa o comando SELECT, é possível completar a cláusula FROM com duas ou mais tabelas que estão relacionadas. No exemplo a seguir, a cláusula FROM terá, além da tabela ITEM_PEDIDO, a tabela PIZZA. As colunas “pizza_id” estão relacionadas inclusive como chave primária e estrangeira.

Deve-se executar o comando SELECT desta nova maneira:

```
mysql> select * from item_pedido i, pizza p;
```

pedido_id	pizza_id	quantidade	valor	pizza_id	nome	descricao	valor
1	1	1	15.00	1	Portuguesa	NULL	15.00
1	1	1	15.00	2	Provolone	NULL	17.00
					...		
8	4	2	34.00	4	Calabresa	NULL	17.00
8	4	2	34.00	5	Escarola	NULL	22.00

Como o comando pede a listagem de todas as colunas, são exibidas as colunas de ITEM_PEDIDO (pedido_id, pizza_id,

quantidade e valor) e as colunas da tabela PIZZA (pizza_id, nome, descrição e valor).

Contudo, como não foi indicado um critério que relacione as duas tabelas, obteve-se o **produto cartesiano** das duas tabelas: 12 linhas da tabela ITEM_PEDIDO vezes 5 linhas da tabela PIZZA resultam em um SELECT com 60 linhas. O pedido número 1 é exibido com os cinco sabores de pizza, assim como os demais. Isso não é útil, pois há informação em excesso e sem significado.

Deve-se então utilizar a cláusula WHERE para determinar que somente serão exibidas as linhas em que o “pizza_id” da tabela ITEM_PEDIDO (i.pizza_id) seja o mesmo “pizza_id” da tabela PIZZA (p.pizza_id). Desse modo, indica-se que o pedido e a pizza devem apontar para a mesma informação. Este é o resultado da relação entre as duas tabelas.

```
select * from item_pedido i, pizza p where i.pizza_id =
p.pizza_id;
```

pedido_id	pizza_id	quantidade	valor	pizza_id	nome	descricao	valor
1	1	1	15.00	1	Portuguesa	NULL	15.00
5	1	3	45.00	1	Portuguesa	NULL	15.00
7	1	2	30.00	1	Portuguesa	NULL	15.00
2	3	2	40.00	3	4 Queijos	NULL	20.00
4	3	1	20.00	3	4 Queijos	NULL	20.00
7	3	1	20.00	3	4 Queijos	NULL	20.00
1	4	1	17.00	4	Calabresa	NULL	17.00
8	4	2	34.00	4	Calabresa	NULL	17.00
3	5	1	22.00	5	Escarola	NULL	22.00
4	5	1	22.00	5	Escarola	NULL	22.00
6	5	2	44.00	5	Escarola	NULL	22.00
7	5	1	22.00	5	Escarola	NULL	22.00

Agora a listagem contém somente informações relacionadas, o que confere significado ao relatório. É possível determinar quais colunas devem ser exibidas, em vez de todas, de modo a tornar a listagem mais simples de ser lida, uma vez que contém somente dados que importam. Por exemplo:

```

select
    i.pedido as "Pedido", p.nome as "Pizza",
    i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
    i.valor as "Total"
from
    item_pedido i, pizza p
where
    i.pizza_id = p.pizza_id
order by i.pedido_id;

```

Pedido	Pizza	Qtde	Unitário	Total
1	Calabresa	1	17.000000	17.00
1	Portuguesa	1	15.000000	15.00
2	4 Queijos	2	20.000000	40.00
3	Escarola	1	22.000000	22.00
4	4 Queijos	1	20.000000	20.00
4	Escarola	1	22.000000	22.00
5	Portuguesa	3	15.000000	45.00
6	Escarola	2	22.000000	44.00
7	Escarola	1	22.000000	22.00
7	Portuguesa	2	15.000000	30.00
7	4 Queijos	1	20.000000	20.00
8	Calabresa	2	17.000000	34.00

Agora se tem quais pizzas foram escolhidas em cada pedido, a quantidade e o valor unitário de cada uma e o valor total do pedido. Vale lembrar que na tabela PIZZA o valor de cada pizza pode sofrer alteração, por isso é feito o cálculo de quanto o cliente pagou por cada pizza no dia em que fez o pedido.

Compreendido o processo básico de associação entre duas ou mais tabelas, é necessário observar que essas consultas se dividem em dois grandes grupos:

- INNER JOIN, quando houver pelo menos uma correspondência nas duas tabelas (como foi feito anteriormente).
- OUTER JOIN, quando não houver ao menos uma correspondência nas duas tabelas, o que pode ser do tipo LEFT JOIN, RIGHT JOIN e FULL JOIN.

INNER JOIN e cláusula WHERE não diferem com relação ao resultado da consulta, mas diferem em relação ao desempenho. A Oracle, proprietária do MySQL, recomenda o uso de INNER JOIN para garantir o melhor desempenho nas consultas que envolvam duas ou mais tabelas.

Cláusula INNER JOIN

Para executar consultas com duas ou mais tabelas relacionadas, utiliza-se o comando SELECT com a cláusula INNER JOIN. Sua sintaxe é:

```
select <colunas>
  from <tabela_1> inner join <tabela_2>
 on <tabela_1.coluna> = <tabela_2.coluna>
 [group by <coluna>]
 [having <condição>]
 [order by <coluna>];
```

A consulta a seguir equivale à que foi feita anteriormente, porém, utilizou-se o INNER JOIN:

```
select
  i.pedido as "Pedido", p.nome as "Pizza",
  i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
  i.valor as "Total"
from
  item_pedido i INNER JOIN pizza p
on
  i.pizza_id = p.pizza_id
order by i.pedido_id;
```

Pedido	Pizza	Qtde	Unitário	Total
1	Calabresa	1	17.000000	17.00
		...		
8	Calabresa	2	17.000000	34.00

A seleção das colunas, o uso de apelidos e a ordenação do resultado da consulta continuam idênticos.

A variação aconteceu no modo como foram selecionadas as tabelas,

```
from item_pedido i INNER JOIN pizza p
on i.pizza_id = p.pizza_id
```

o que indica que a tabela ITEM_PEDIDO se relacionará com a tabela PIZZA, equivalente à cláusula FROM usada anteriormente, que lista somente as linhas em que o código da pizza seja o mesmo em ambas as tabelas.

Cláusula LEFT JOIN

Como parte das cláusulas OUTER JOIN que listam dados sem correspondência na relação, o LEFT JOIN retorna todas as linhas da tabela 1 (à esquerda) combinadas com suas correspondentes na tabela 2 (da direita) ou com resultado NULL caso essa correspondência não exista.

Sua sintaxe é:

```
select <colunas>
  from <tabela_1> left join <tabela_2>
    on <tabela_1.coluna> = <tabela_2.coluna>
  [group by <coluna>]
  [having <condição>]
  [order by <coluna>];
```

Para que uma pizza seja vendida, é necessário que ela esteja cadastrada. Contudo, existe a possibilidade de uma pizza do cadastro nunca ter sido vendida. No exemplo a seguir, a tabela 1 equivale a “pizza” e a tabela 2 equivale a “item_pedido” e serão listadas todas as pizzas, mas se alguma nunca foi vendida, os dados do pedido serão preenchidos com NULL:

```
select
    i.pedido as "Pedido", p.nome as "Pizza",
    i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
    i.valor as "Total"
from
    pizza p LEFT JOIN item_pedido i
on
    p.pizza_id = i.pizza_id
order by i.pedido_id;
```

Pedido	Pizza	Qtde	Unitário	Total
NULL	Provolone	NULL	NULL	NULL
	...			
8	Calabresa	2	17.000000	34.00

O resultado indica que a pizza de provolone está cadastrada, mas nunca foi vendida.

Em consultas OUTER JOIN é comum adicionar a cláusula WHERE para aplicar um filtro. Se o objetivo é listar somente as pizzas que nunca foram vendidas, pode-se testar se o número do pedido é igual a NULL, como indicado a seguir:

```
select
    i.pedido as "Pedido", p.nome as "Pizza",
    i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
    i.valor as "Total"
from
    pizza p LEFT JOIN item_pedido i
on
    i.pedido = NULL;
```

```

on
  p.pizza_id = i.pizza_id
where i.pedido_id is null;
+-----+-----+-----+-----+
| Pedido | Pizza      | Qtde | Unitário   | Total |
+-----+-----+-----+-----+
| NULL   | Provolone  | NULL | NULL       | NULL  |
+-----+-----+-----+-----+

```

Cláusula RIGHT JOIN

Como parte das cláusulas OUTER JOIN que listam dados sem correspondência na relação, o RIGHT JOIN retorna todas as linhas da tabela 2 (à direita) combinadas com suas correspondentes na tabela 1 (da esquerda) ou com resultado NULL, caso essa correspondência não exista. Isso evidencia que o RIGHT JOIN tem comportamento oposto ao LEFT JOIN.

Sua sintaxe é:

```

select <colunas>
  from <tabela_1> right join <tabela_2>
    on <tabela_1.coluna> = <tabela_2.coluna>
 [group by <coluna>]
 [having <condição>]
 [order by <coluna>];

```

A seguir, será executada a mesma consulta do LEFT JOIN, adequada para o RIGHT JOIN:

```

select i.pedido_id as "Pedido", p.nome as "Pizza",
       i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
       i.valor as "Total"
  from item_pedido i right join pizza p
    on i.pizza_id = p.pizza_id
    order by i.pedido_id;

```

Pedido	Pizza	Qtde	Unitário	Total
NULL	Provolone	NULL	NULL	NULL
8	Calabresa	2	17.000000	34.00

Novamente, é possível adicionar uma cláusula WHERE e obter como resultado somente as linhas da tabela da direita sem equivalente na tabela da esquerda:

```
select
    i.pedido as "Pedido", p.nome as "Pizza",
    i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
    i.valor as "Total"
from
    item_pedido i RIGHT JOIN pizza p
on
    i.pizza_id = p.pizza_id
where i.pedido_id is null;
```

RECAPITULANDO

Executar consultas, de modo a determinar filtros, agrupamentos e ordenação para a resposta, pode não ser suficiente para atender às regras de negócio que se deseja implementar. Às vezes, pode ser necessário mesclar dados de duas ou mais tabelas para produzir a resposta que se deseja.

Ao mesclar duas tabelas, pode-se determinar o que será adicionado ou removido na resposta da pesquisa, o que pode funcionar como ferramenta de análise de dados.

É de grande valia para o programador saber utilizar os JOINS de forma adequada, para delegar ao servidor de dados atividades que consumam grande processamento, de modo a fazer com que o programa instalado em um micro ou servidor de aplicações seja executado de maneira mais eficiente por receber os dados já processados.

Exercícios

1. A partir da utilização do INNER JOIN, criar a consulta que exiba as colunas número do pedido, data do pedido, cliente que fez o pedido. A consulta deve estar classificada em ordem crescente pelo número do pedido.

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

7. Índices e visões

Comando CREATE VIEW

Comando ALTER VIEW

Comando DROP VIEW

Comando CREATE INDEX

Comando DROP INDEX

Ao realizar um SELECT a partir da aplicação de um filtro com a cláusula WHERE, o MySQL pesquisará linha a linha da tabela, de modo a realizar a comparação do valor do filtro até encontrar o valor desejado ou retornar zero linha de resposta.

Esse processo é lento e consome grandes recursos do servidor de dados.

Para evitar situações como essa, as colunas que são frequentemente consultadas podem ser indexadas, para que a pesquisa ocorra de modo eficiente no arquivo de índice e, ao encontrar o valor, aponte para o arquivo de dados na posição correspondente.

Desse modo, serão consumidos poucos recursos do servidor de dados, o que faz com que as pesquisas sejam mais eficientes.

Quando se realiza um SELECT muito complexo, com diversas cláusulas, testes e ordenações para produzir um resultado específico, pode-se criar VIEWS (visões, em inglês). Elas representam uma visão dos dados, não sendo os dados propriamente ditos. Por essa razão, são conhecidas como tabelas virtuais.

Este capítulo aborda como:

- criar e excluir visões;
- criar e excluir índices.

Comando CREATE VIEW

Para criar uma visão em um banco de dados, utiliza-se o comando CREATE VIEW. Sua sintaxe é:

```
create view <nome> [(<colunas>)] as <pesquisa>;
```

“Nome” é o título da visão, equivale ao nome de uma tabela. As “colunas” são os apelidos que serão dados às colunas resultantes da pesquisa. A “pesquisa” é o comando SELECT.

No capítulo anterior, criou-se a consulta de pizzas sem movimentação. É um comando grande, mas, se gravado em uma VIEW, torna-se simples de ser utilizado:

```
create view pizza_sem_movimento (pedido, pizza, qtde,
    unitario, total)
as
select i.pedido_id as "Pedido", p.nome as "Pizza",
    i.quantidade as "Qtde", (i.valor/i.quantidade) as
"Unitário",
    i.valor as "Total"
    from pizza p left join item_pedido i
    on p.pizza_id = i.pizza_id
    where i.pedido_id is null;
```

Após o comando, será criada uma consulta chamada “pizza_sem_movimento”, que contém as colunas pedido, pizza, quantidade, unitário e total. Toda vez que for executado um SELECT nessa consulta, ela será preenchida com os dados da pesquisa criada na visão. O comando SELECT utilizado na criação da visão pode ser gerado com todas as possibilidades estudadas anteriormente: filtros, ordenações, agrupamentos, *joins* etc.

Para ver o resultado de uma VIEW, executa-se o SELECT:

```
select * from pizza_sem_movimento;
```

pedido	pizza	qtde	unitario	total
NULL	Provolone	NULL	NULL	NULL

Nota-se que a consulta ficou bastante simplificada, o que facilitou o dia a dia do programador que for utilizá-la.

Comando ALTER VIEW

Para alterar uma visão em um banco de dados, utiliza-se o comando ALTER VIEW. Sua sintaxe é:

```
alter view <nome> [(<colunas>)] as <pesquisa>;
```

O “nome” da visão deve existir para que as “colunas” e a “pesquisa” possam ser alteradas.

Para demonstrar o comando, será alterada a visão “pizza_sem_movimento” para ter somente as colunas “pizza_id” e “nome”, e não exibir colunas com valor NULL:

```
alter view pizza_sem_movimento (pizza_id, nome)
as
select p.pizza_id as "pizza_id", p.nome as "nome"
from pizza p left join item_pedido i
on p.pizza_id = i.pizza_id
where i.pedido_id is null;
```

Ao executar a pesquisa na visão, a resposta será:

```
select * from pizza_sem_movimento;
+-----+-----+
| pizza_id | nome      |
+-----+-----+
|          2 | Provolone |
+-----+-----+
```

Comando DROP VIEW

Para excluir uma visão em um banco de dados, utiliza-se o comando DROP VIEW. Sua sintaxe é:

```
drop view <nome>;
```

Para excluir a visão “pizza_sem_movimento”:

```
drop view pizza_sem_movimento;
```

Se for executado um SELECT na visão, após sua exclusão, será retornada a mensagem de que a tabela não existe, pois visões e tabelas são tratadas de modo idêntico nas pesquisas.

Comando CREATE INDEX

Para criar um índice de coluna em um banco de dados, utiliza-se o comando CREATE INDEX. Sua sintaxe é:

```
create [unique] index <nome> on <tabela> (<colunas>;
```

Se o índice for criado com UNIQUE, não serão permitidos dados repetidos na coluna. “Nome” é o nome que se dá ao índice e “tabela” é qual tabela contém as “colunas” que serão indexadas.

Por exemplo, para criar índice para a coluna “nome” da tabela “cliente”:

```
create index idx_nome on cliente (nome);
```

Outro exemplo, para criar índice exclusivo (sem repetição) para a coluna “nome” da tabela cliente:

```
create unique index idx_nome on cliente (nome);
```

Existem duas situações a serem consideradas no uso da cláusula UNIQUE: se a tabela estiver vazia, todo INSERT e UPDATE só será concluído se a exclusividade do índice for respeitada, ou seja, se eles não tentarem gravar um nome que já existe; se a tabela já contém dados, eles serão analisados antes da conclusão da criação do índice e, se existirem nomes repetidos, o índice não será criado.

Comando DROP INDEX

Para excluir um índice de um banco de dados, utiliza-se o comando DROP INDEX. Sua sintaxe é:

```
drop index <nome> on <tabela>;
```

Para excluir o índice idx_nome da tabela “cliente”:

```
drop idx_nome on cliente;
```

RECAPITULANDO

Com o objetivo de melhorar o desempenho de um banco de dados, pode-se utilizar os índices nas colunas que são utilizadas como filtros em pesquisas.

É um recurso que deve ser utilizado com moderação, especialmente em colunas que terão os dados constantemente alterados, pois, nesse caso, além da alteração na tabela, o índice deverá ser reconstruído. Isso causaria o efeito inverso do desejado, uma vez que haveria degradação no desempenho do banco.

Outro recurso bastante útil é a criação de visões ou “tabelas virtuais”. Além da facilidade de resumir um SELECT complexo em um único comando, já que este será escrito apenas uma vez e invocado quantas vezes for necessário, outra característica da visão é relacionada à segurança dos dados por não haver acesso à tabela ou às colunas reais.

Exercícios

1. Toda vez que um cliente liga para fazer um pedido, o número do telefone é exibido no identificador de chamadas e o atendente deverá buscá-lo no cadastro de clientes. Isso implica em realizar diversas buscas na coluna telefone. Criar um índice exclusivo para essa coluna de modo a garantir a melhor *performance* nessas consultas.

2. O proprietário necessita de um relatório gerencial chamado “Curva ABC de Venda”. Nesse relatório, é listado o nome da pizza e quantas delas foram vendidas, em ordem decrescente de quantidade. Criar uma VIEW chamada “abc_de_venda” que gere essa listagem.

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

8. Funções, procedimentos e gatilhos

Funções
Programação de funções
Procedimentos
Gatilhos

Em MySQL, é possível criar blocos de códigos reutilizáveis, conhecidos como funções e procedimentos. As funções e os procedimentos são usados para evitar a repetição de códigos, de maneira a reaproveitar e facilitar sua manutenção. Uma vez criado o bloco de códigos, ele poderá ser utilizado em toda a base de dados. Caso seja definida a necessidade de alterar a maneira como o código é executado, basta alterar esse bloco e todas as chamadas a ele estarão de acordo com essa nova versão.

Essas funções e esses procedimentos podem ser utilizadas para executar algum cálculo ou verificação e devolver uma resposta ao comando que a utiliza.

O objetivo deste capítulo é apresentar de forma básica a criação e manutenção de funções e procedimentos para leitores conhecedores dos princípios de lógica de programação e, preferencialmente, de alguma linguagem de programação.

Este capítulo aborda como:

- criar e excluir funções;
- criar e excluir procedimentos;
- consumir funções e procedimentos;
- criar e excluir gatilhos (*triggers*).

Funções

Funções são partes do código invocadas com ou sem parâmetro que devolvem um valor como resposta de sua execução. No MySQL, as funções são conhecidas como *stored functions*.

Os nomes de função são seguidos de parênteses nos quais serão informados os parâmetros, caso existam. Esses parênteses são obrigatórios mesmo na ausência de parâmetros.

No Capítulo 4, foi estudada a função `now()`, que devolve a data e a hora atual do servidor MySQL. Essa é uma função que não recebe parâmetros. No mesmo capítulo, estudou-se a função `year(<data>)` que devolve o ano da data informada como parâmetro. Como exemplo, pode-se digitar:

```
select year(now()) as "ano";
```

Comando DROP FUNCTION

Para remover uma função do banco de dados, utiliza-se o comando `DROP FUNCTION`. Sua sintaxe é:

```
drop function [if exists] <nome>;
```

Drop é a palavra reservada do SQL que indica remoção de um objeto; *function* é o tipo de objeto que se deseja remover, neste caso, uma função; a cláusula *if exists* é opcional e determina que o comando será executado somente se a função <nome> existir; nome é a função que se deseja remover.

Por exemplo, para remover a função CUMPRIMENTO, que será criada posteriormente, digitar:

```
drop function if exists cumprimento;
```

Comando ALTER FUNCTION

O comando ALTER FUNCTION é utilizado quando se deseja alterar as características de uma função, porém, no MySQL não é possível alterar o corpo da função com esse comando. Por essa razão, ele não será discutido neste livro.

Comando CREATE FUNCTION

Para criar uma função, utiliza-se o comando CREATE FUNCTION. Sua sintaxe é:

```
create function
    <nome>([parâmetro_1 <tipo>], [parâmetro n <tipo>])
    returns <tipo>
begin
    <comando 1>;
    <comando n>;
    return <valor>;
end;
```

Antes de iniciar com exemplos, observar que durante a criação de uma função o ponto e vírgula é usado diversas vezes, sendo ele o delimitador do MySQL, ou seja, o caractere que determina se um comando foi inserido por completo e deve ser executado.

Para evitar isso, é necessário alterar o delimitador para outro caractere, digitar o comando para a criação da função e voltar o delimitador para o padrão, que é o ponto e vírgula. Isso é feito com o comando DELIMITER, conforme demonstrado a seguir:

```
delimiter //  
select * from pizza//  
delimiter $$  
select * from pizza$$  
delimiter ;  
select * from pizza;
```

O primeiro comando DELIMITER altera o delimitador para “//”. Normalmente, ao utilizar um delimitador diferente do padrão, opta-se por caracteres duplos e repetidos. Isso não é regra, mas uma boa prática no uso de banco de dados.

Para exemplificar a mudança do delimitador, foi executado um SELECT na tabela PIZZA.

O segundo comando DELIMITER altera o delimitador para “\$\$” e, novamente, foi executado um SELECT para exemplificar seu uso.

O terceiro comando DELIMITER altera o delimitador para “;” – que é o padrão. O último SELECT é executado após o ponto e vírgula, como tem sido feito desde o início do livro.

Para criar uma função que não recebe parâmetros e tem um texto como retorno, deve-se executar os comandos conforme indicado a seguir, com base nas respostas devolvidas:

```
delimiter //  
create function cumprimento() returns text  
begin  
return “Olá!”;  
end//  
Query OK, 0 rows affected (0.00 sec)  
delimiter ;
```

O delimitador foi alterado para “//”, para dar início à digitação do comando de criação da função. Ao final, o comando END é seguido do “//”, que indica que a digitação acabou e o comando deve ser executado. A resposta “Query OK” indica que não houve erros e a função foi gravada com sucesso. Para concluir, o delimitador deve ser alterado para “;”.

Pode-se executar a função criada por meio do comando SELECT:

```
select cumprimento();
+-----+
| cumprimento() |
+-----+
| Olá!          |
+-----+
select cumprimento(), nome from cliente;
+-----+-----+
| cumprimento() | nome          |
+-----+-----+
| Olá!          | Alexandre Santos |
|               | ...            |
| Olá!          | Laura Madureira  |
+-----+-----+
```

A função também poderá ser executada em qualquer comando que necessite de um texto, por exemplo, em um INSERT:

```
insert into pizza (nome, valor) values (cumprimento(), 10.0);
```

Nesse caso, será criada uma nova linha na tabela PIZZA com o nome “Olá!” e o valor R\$ 10,00.

Para criar uma função que receba um parâmetro, é necessário informar o nome da variável que receberá o valor do parâmetro e o tipo dessa variável. Caso seja usado mais de um parâmetro, eles devem ser separados por vírgula.

O bloco de comandos a seguir vai excluir e recriar a função CUMPRIMENTO para receber um texto como parâmetro e devolver como resposta a concatenação da palavra “Olá,” com este texto:

```
drop function if exists cumprimento;
create function cumprimento(nome text) returns text
begin
return concat("Olá, ", nome);
end//
Query OK, 0 rows affected (0.00 sec)
delimiter ;
```

Para testar o novo comportamento da função CUMPRIMENTO, usa-se:

```
select cumprimento(nome) from cliente;
+-----+
| cumprimento(nome) |
+-----+
| Olá, Alexandre Santos |
|           ...         |
| Olá, Laura Madureira  |
+-----+
```

Também é possível a criação de variáveis no corpo da função. O comando DECLARE é utilizado para declarar o nome e o tipo de uma variável, enquanto o comando SET é usado para atribuir um valor a uma variável declarada.

O bloco de comandos a seguir exclui e recria a função CUMPRIMENTO, que agora utiliza uma variável chamada “mensagem” para armazenar o resultado da concatenação do cumprimento com o nome obtido na tabela CLIENTE. Esta variável será utilizada no comando RETURN. Ao executar o teste anterior, obtém-se o mesmo resultado, porém, o processamento da função é diferente do criado anteriormente.

Para finalizar este tópico, será criada uma função que recebe uma data de nascimento e determina a idade do indivíduo em relação à data do servidor:

```
drop function if exists nascimento;
delimiter //
create function nascimento(dataNascimento date) returns
int
begin
  declare idade int;
  if date_format(now(), "00-%m-%d") >= date_
format(nascimento, "00-%m-%d") then
    set idade = year(now()) - year(nascimento);
  else
    set idade = year(now()) - year(nascimento) - 1;
  end if;
  return idade;
end//
Query OK, 0 rows affected (0.00 sec)
delimiter ;
```

Qual seria a idade de Steve Jobs, nascido em 24 de fevereiro de 1955? (A data do servidor no qual o comando foi executado era 30/01/2017):

```
select nascimento("1955-02-24") as "Idade de Steve Jobs";
+-----+
| Idade de Steve Jobs |
+-----+
|                    62 |
+-----+
```

Programação de funções

Criar funções para uso no MySQL implica, além dos conhecimentos de SQL, conhecimentos de programação, como criação de variáveis, uso de operadores aritméticos, relacionais e lógicos, estruturas de decisão e repetição.

Declaração de variáveis

O comando **DECLARE** é usado para a declaração da variável. Sua sintaxe é:

```
declare <nome 1>, <nome 2>, <nome n> <tipo>;
```

Pode haver uma ou mais linhas com o comando **DECLARE**, mas todas devem estar antes de outros comandos, ou seja, as variáveis devem ser declaradas no início do corpo da função.

Todos os tipos de dados do MySQL podem ser utilizados como tipos de variáveis. Por exemplo:

```
declare nome varchar(30);
declare peso float;
declare altura double;
declare imc numeric(5,2);
```

Operadores aritméticos

Os operadores aritméticos são usados para executar cálculos, como adição, subtração, multiplicação, divisão e resto da divisão. O resultado de uma operação aritmética é o valor do resultado da operação. O Quadro 1 apresenta os operadores aritméticos.

Quadro 1 – Operadores aritméticos

Nome	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
DIV	Divisão de inteiros
% ou MOD	Resto da divisão

Operadores relacionais

Os operadores relacionais são usados para comparar dois valores. O resultado de uma operação relacional é 1 (um) se a comparação for verdadeira ou 0 (zero) se a comparação for falsa. O Quadro 2 apresenta os operadores relacionais.

Quadro 2 – Operadores relacionais

Nome	Descrição
>	Maior
>=	Maior ou igual
<	Menor
<=	Menor ou igual
=	Igual
<=>	Igual para nulos
<> ou !=	Diferente

Operadores lógicos

Os operadores lógicos são usados para criar expressões lógicas. O resultado de uma expressão lógica é 1 (um) se a expressão for verdadeira ou 0 (zero) se a expressão for falsa. Os operadores lógicos estão listados no Quadro 3.

Quadro 3 – Operadores lógicos

Nome	Descrição
And ou &&	E
OR ou	Ou
NOT ou !	Negação
XOR	Ou exclusivo

Estrutura de decisão

Também chamadas de estruturas condicionais, são utilizadas em conjunto com operadores relacionais e lógicos para a tomada de decisão após o resultado de um teste. Sua sintaxe é:

```
if <teste> then <comandos>
  [elseif <teste> then <comandos>] ...
  [else <comandos>]
end if
```

O exemplo a seguir recebe dois valores inteiros e devolve como resposta um texto que determina se o primeiro é maior, menor ou igual ao segundo:

```
drop function if exists testeSimples;
delimiter //
create function testeSimples(a int, b int) returns
varchar(20)
begin
  -- A e B são os valores e R a resposta do teste
  declare r varchar(20);
  if a > b then
    set r = ">";
  elseif a = b then
    set r = "=";
  else
    set r = "<";
  end if;
  set r = concat(a, " ", r, " ", b);
  return r;
end //
delimiter ;
```

Para testar a função, utilizam-se os exemplos apresentados no Quadro 4.

Quadro 4 – Exemplos de uso da função teste

Teste	Resposta
select testeSimples(10, 20);	10 < 20
select testeSimples(20, 10);	20 > 10
select testeSimples(10, 10);	10 = 10

O código a seguir apresenta variações em relação ao código anterior, tanto na lógica quanto na resposta:

```
drop function if exists testeSimples;
delimiter //
create function testeSimples(a int, b int) returns
varchar(50)
begin
declare r varchar(50);
if a = b then
    set r = "é igual a";
else
    if a > b then
        set r = "maior";
    else
        set r = "menor";
    end if;
set r = concat("é ", r, " do que");
end if;
set r = concat(a, " ", r, " ", b, ".");
return r;
end //
delimiter ;
```

Para testar a função, utilizam-se os exemplos indicados no Quadro 5.

Quadro 5 – Exemplos de uso da função teste

Teste	Resposta
select testeSimples(10, 20);	10 é menor que 20
select testeSimples(20, 10);	20 é maior que 10
select testeSimples(10, 10);	10 é igual a 10

Além do comando IF, existe a estrutura de decisão CASE. Suas sintaxes são:

```
case <variavel>
  when <valor> then <comandos> ...
  [else <comandos>]
end case
```

```

case
  when <teste> then <comandos> ...
  [else <comandos>]
end case

```

No exemplo a seguir, a função “nomeDoMes” recebe um número inteiro como parâmetro e devolve o nome do mês informado ou inválido, caso não se trate de um número de mês. Uma das sintaxes do comando CASE é usada para executar uma sequência de testes no mesmo valor, o que facilita a programação.

```

drop function if exists nomeDoMes;
delimiter //
create function nomeDoMes(mes tinyint) returns varchar(10)
begin
declare nomeDoMes varchar(10);
case mes
  when 1 then set nomeDoMes = "janeiro";
  when 2 then set nomeDoMes = "fevereiro";
  when 3 then set nomeDoMes = "março";
  when 4 then set nomeDoMes = "abril";
  when 5 then set nomeDoMes = "maio";
  when 6 then set nomeDoMes = "junho";
  when 7 then set nomeDoMes = "julho";
  when 8 then set nomeDoMes = "agosto";
  when 9 then set nomeDoMes = "setembro";
  when 10 then set nomeDoMes = "outubro";
  when 11 then set nomeDoMes = "novembro";
  when 12 then set nomeDoMes = "dezembro";
  else set nomeDoMes = "Inválido";
end case;
return nomeDoMes;
end //
delimiter ;

```

No próximo exemplo, a função “diaDaSemana” devolve o nome do dia da semana da data do servidor. Uma das sintaxes do comando CASE é usada para executar uma sequência de testes em uma variável, o que facilita a programação.

```
drop function if exists diaDaSemana;
delimiter //
create function diaDaSemana() returns varchar(15)
begin
declare numeroDoDia tinyint;
declare nomeDoDia varchar(15);
set numeroDoDia = dayofweek(now());
case
when numeroDoDia = 1 then set
    nomeDoDia = "domingo";
when numeroDoDia = 2 then set
    nomeDoDia = "segunda-feira";
when numeroDoDia = 3 then set
    nomeDoDia = "terça-feira";
when numeroDoDia = 4 then set
    nomeDoDia = "quarta-feira";
when numeroDoDia = 5 then set
    nomeDoDia = "quinta-feira";
when numeroDoDia = 6 then set
    nomeDoDia = "sexta-feira";
when numeroDoDia = 7 then set
    nomeDoDia = "sabado";
end case;
return nomeDoDia;
end //
delimiter ;
```

Estruturas de repetição

Para repetir um bloco de comandos, utiliza-se uma das três estruturas de repetição existentes no MySQL: LOOP, WHILE ou REPEAT. A sintaxe do comando de repetição LOOP é:

```
<nome do loop>:loop
  <comandos>
  leave <nome do loop>;
  iterate <nome do loop>;
end loop <nome do loop>;
```

O “nome do loop” é um texto que identifica o laço. O comando LEAVE é utilizado para interromper o laço, seguindo o programa após a linha da declaração END LOOP, enquanto o comando ITERATE é utilizado para retornar à linha da declaração LOOP ao início do laço.

O exemplo devolve como resposta a soma dos números pares entre os valores inicial e final, informados como parâmetros da função:

```
drop function if exists testeLoop;
delimiter //
create function testeLoop(inicial int, final int) returns
int
begin
  declare cnt, total int;
  set cnt = inicial;
  set total = 0;
  if inicial < final then
    contador:loop
      set cnt = cnt + 1;
      if cnt%2 = 0 then
        set total = total + cnt;
      end if;
      if cnt = final then
        leave contador;
      end if;
    end loop contador;
  end if;
  return total;
end //
delimiter ;
```

A sintaxe do comando de repetição WHILE é:

```
<nome do loop>:while <condição> do
  <comandos>
end while <nome do loop>;
```

Na estrutura de repetição WHILE, enquanto a condição for verdadeira, o laço se repetirá. Os comandos LEAVE e ITERATE também podem ser usados nessa estrutura.

O exemplo a seguir é a versão com comando WHILE da função escrita no tópico anterior:

```
drop function if exists testeWhile;
delimiter //
```

```
create function testeWhile(inicial int, final int) returns
int
begin
    declare cnt, total int;
    set cnt = inicial;
    set total = 0;
    if inicial < final then
        contador:while cnt <= final do
            if cnt%2 = 0 then
                set total = total + cnt;
            end if;
            set cnt = cnt + 1;
        end while contador;
    end if;
    return total;
end //
delimiter ;
```

A sintaxe do comando de repetição REPEAT é:

```
<nome do loop>:repeat
    <comandos>
until <condição>
end repeat <nome do loop>;
```

Na estrutura de repetição REPEAT, o laço se repetirá até que a condição seja satisfeita. Os comandos LEAVE e ITERATE também podem ser usados nessa estrutura.

O exemplo a seguir é a versão com comando REPEAT da função escrita no tópico anterior:

```
drop function if exists testeRepeat;
delimiter //
create function testeRepeat(inicial int, final int)
returns int
begin
    declare cnt, total int;
    set cnt = inicial;
    set total = 0;
    if inicial < final then
        contador:repeat
```

```

        if cnt%2 = 0 then
            set total = total + cnt;
        end if;
        set cnt = cnt + 1;
        until cnt > final
        end repeat contador;
    end if;
    return total;
end //
delimiter ;

```

Procedimentos

Assim como as funções, os procedimentos são partes do código invocadas com ou sem parâmetro, mas que não devolvem um valor como resposta de sua execução. No MySQL, os procedimentos são conhecidos como *stored procedures*.

No corpo do procedimento, no qual ficam as instruções, pode-se utilizar comandos que exibem dados na tela, como SELECT. Outro detalhe importante é que, para determinar constantes caracteres, utiliza-se a aspa simples (apóstrofo) em vez de aspas.

Comando DROP PROCEDURE

O comando DROP PROCEDURE é usado para remover um procedimento do banco de dados. Sua sintaxe é:

```
drop procedure [if exists] <nome>;
```

Por exemplo, para remover o procedimento “totalDePizzas” que será criado posteriormente:

```
drop procedure if exists totalDePizzas;
```

Comando ALTER PROCEDURE

O comando ALTER PROCEDURE é utilizado quando se deseja alterar as características de um procedimento, porém, não é possível alterar o corpo do procedimento com esse comando. Por essa razão, ele não será discutido neste livro.

Comando CREATE PROCEDURE

O comando CREATE PROCEDURE é usado para criar um procedimento. Sua sintaxe é:

```
create procedure
    <nome>([{IN | OUT | INOUT} parâmetro_1 <tipo>],
          [{IN | OUT | INOUT} parâmetro_n <tipo>])
begin
    <comando 1>;
    <comando n>;
end;
```

A declaração dos parâmetros deve ser precedida de um qualificador que determine se o parâmetro será utilizado para receber informações (IN), para enviar informações (OUT) ou para ambos (INOUT).

O exemplo a seguir demonstra a criação de um procedimento sem parâmetros:

```
drop procedure if exists hoje;
delimiter //
create procedure hoje()
begin
    select date_format(now(), "%d/%m/%Y") as 'Hoje';
end //
delimiter ;
```

Para executar um procedimento, utiliza-se o comando CALL:


```
call hoje;
+-----+
| Hoje   |
+-----+
| 02/02/2017 |
+-----+
```

É possível criar procedimentos que recebam parâmetros em sua chamada. Os parâmetros podem ser de três tipos:

- **IN:** determina que o parâmetro é utilizado para receber valores.
- **OUT:** determina que o parâmetro é utilizado para devolver valores.
- **INOUT:** determina que o parâmetro é utilizado tanto para receber quanto para devolver valores.

Os exemplos a seguir ilustram o uso dos três tipos de parâmetros.

O procedimento “partesDoTempo” deve receber um caractere como parâmetro. Se o caractere for “h” ou “H” será exibida a hora, se for “m” ou “M”, será exibido o minuto, e se for “s” ou “S”, será exibido o segundo. Qualquer informação se refere à hora do servidor no momento da consulta.

```
drop procedure if exists partesDoTempo;
delimiter //
create procedure partesDoTempo(in parte char)
begin
    case parte
        when 'h' then select date_format(now(), "%H") as
'Hora';
        when 'm' then select date_format(now(), "%i") as
'Minuto';
        when 's' then select date_format(now(), "%s") as
'Segundo';
        else select "Opcao invalida" as 'Erro';
    end case;
end //
delimiter ;
```

O Quadro 6 mostra exemplos de retorno do procedimento.

Quadro 6 – Exemplos de uso do procedimento partesDoTempo

Chamada	Retorno
call partesDoTempo('h');	21
call partesDoTempo('m');	3
call partesDoTempo('s');	58
call partesDoTempo('a');	Opção inválida

Um procedimento pode utilizar parâmetros anotados como OUT para externar valores utilizados internamente. Observe-se que não há contradição entre o que foi escrito anteriormente (*mas que não devolvem um valor como resposta de sua execução*) por não haver retorno de valor. O valor de um parâmetro OUT fica armazenado em uma variável. Isso pode ser feito de forma explícita (`set @<nome> = <valor>;`) ou de forma implícita (`call <procedimento>(@<variável>;)`), e o valor da variável pode ser consultado pelo comando `SELECT @<variável>;`.

Serão promovidas alterações no procedimento “partesDoTempo” para guardar o valor do retorno num parâmetro do tipo OUT em vez de exibir o resultado na tela. Se a parte do tempo enviada no parâmetro “parte” não for reconhecida, a variável de retorno armazenará -1.

```
drop procedure if exists partesDoTempo;
delimiter //
create procedure partesDoTempo(in parte char, out resposta
tinyint)
begin
    case parte
    when 'h' then set resposta = hour(now());
    when 'm' then set resposta = minute(now());
    when 's' then set resposta = second(now());
```

```

        else set resposta = -1;
    end case;
end //
delimiter ;

```

Para chamar um procedimento com variáveis de retorno, é necessário informar o nome da variável que se deseja utilizar precedido de @. Variáveis criadas dessa maneira permanecem na sessão do MySQL e, quando o console for encerrado, essas variáveis serão perdidas. O exemplo a seguir foi executado às 8h34.

```

select @retorno;
+-----+
| @retorno |
+-----+
| NULL    |
+-----+
call partesDoTempo('m', @retorno);
select @retorno;
+-----+
| @retorno |
+-----+
|         34 |
+-----+

```

A variável “retorno” não continha valor, mas, após a execução do procedimento, recebeu o valor dos segundos da hora daquele momento.

Quando um parâmetro é anotado como INOUT, pode ser utilizado para receber valor da chamada do procedimento e ser utilizado para devolução de valor a uma variável.

Uma observação relevante não tratada até o momento é sobre o uso dos comandos DML, em especial o SELECT, em funções e procedimentos.

```

drop procedure if exists totalDePizzas;
delimiter //
create procedure totalDePizzas(out total int)
begin

```

```
set total = (select sum(quantidade) from item_pedido);  
end //  
delimiter ;
```

O procedimento “totalDePizzas” executa o comando SELECT e o valor retornado é armazenado na variável total. Conclui-se que tudo o que foi estudado sobre o comando SELECT pode ser utilizado em um procedimento, o que maximiza sua utilidade.

Gatilhos

Existem situações em que um procedimento deve ser executado de forma implícita, disparado a partir de um comando. O nome dessa característica, gatilho (*trigger*, em inglês), justifica-se pelo verbo usado para defini-la, “disparar”.

Para exemplificar essa característica, será necessário criar a tabela de funcionários da pizzeria. Toda vez que um funcionário executar uma ação – cadastrar ou alterar os dados de um usuário, cadastrar um novo pedido, por exemplo – o sistema deverá registrar qual funcionário executou a ação.

```
create table log (  
    data_hora datetime default current_timestamp,  
    tabela varchar(30) not null,  
    atividade char(3) not null,  
    id int not null  
);
```

A ideia é registrar a data e hora em que ocorreu uma ação, o nome da tabela sobre a qual essa ação foi executada, o tipo da ação (INC para inclusão, ALT para alteração e EXC para exclusão).

Comando CREATE TRIGGER

O comando CREATE TRIGGER é usado para criar um gatilho. Sua sintaxe é:

```
create trigger
  <nome> <quando> <acao> on <tabela> for each row
begin
  <comando 1>;
  <comando n>;
end;
```

“Nome” é o nome do gatilho e “quando” determina se o gatilho é disparado BEFORE (antes) ou AFTER (após) a “ação” ser concluída. A “ação” é representada pelos comandos INSERT, DELETE e UPDATE.

O código a seguir cria um gatilho que será disparado toda vez que houver uma inclusão na tabela “cliente”:

```
delimiter $$
create trigger cliente_inc after insert on cliente
  for each row begin
  insert into log values
    (current_timestamp(), «cliente», «INC», new.cliente_
id);
  end $$
delimiter ;
```

Para testar essa TRIGGER, deve-se incluir um cliente e executar um SELECT na tabela log:

```
insert into cliente
  (cliente_id, nome, telefone, endereco, bairro)
values
  (7, "Judite Alves", "1234-5678",
  "Rua dos Pinheiros, 777", "Jardins");
select * from log;
```

data_hora	tabela	atividade	id
2017-01-30 22:08:40	cliente	INC	7

O código a seguir cria um gatilho que será disparado toda vez que houver uma alteração na tabela “cliente”:

```
delimiter $$
create trigger cliente_alt after update on cliente
  for each row begin
    insert into log values
      (current_timestamp(), "cliente", "ALT", new.cliente_
id);
  end $$
delimiter ;
```

Para testar essa TRIGGER, deve-se alterar um cliente e executar um SELECT na tabela log:

```
update cliente set telefone = "8765-4321" where cliente_id
= 7;
select * from log;
```

data_hora	tabela	atividade	id
2017-01-30 22:08:40	cliente	INC	7
2017-01-31 21:28:31	cliente	ALT	7

O código a seguir cria um gatilho que será disparado toda vez que houver uma exclusão na tabela “cliente”:

```
delimiter $$
create trigger cliente_exc after delete on cliente
  for each row begin
    insert into log values
      (current_timestamp(), "cliente", "EXC", old.cliente_
id);
  end $$
delimiter ;
```

Para testar essa TRIGGER, deve-se excluir um cliente e executar um SELECT na tabela log:

```
delete from cliente where cliente_id = 7;
select * from log;
```

data_hora	tabela	atividade	id
2017-01-30 22:08:40	cliente	INC	7
2017-01-31 21:28:31	cliente	ALT	7
2017-01-31 21:42:00	cliente	EXC	7

Note-se a possibilidade de utilizar dados da linha de comandos que dispara o TRIGGER por meio dos prefixos OLD (antes da execução do comando) e NEW (após a execução do comando), seguido do nome da coluna que se deseja utilizar.

Comando DROP TRIGGER

O comando DROP TRIGGER é usado para excluir um gatilho. Sua sintaxe é:

```
drop trigger <nome>;
```

O exemplo a seguir exclui o gatilho criado para registrar as exclusões dos clientes:

```
drop trigger cliente_exc;
```

RECAPITULANDO

Neste capítulo, foram apresentadas as funções e os procedimentos, elementos de grande importância para a customização do banco de dados.

Rotinas específicas da regra de negócios que estão em processo de implementação poderão ser programadas, de modo a facilitar o trabalho dos programadores de linguagem que utilizarão esse banco, o que garantirá boa *performance* na execu-

ção das operações, já que serão executadas em um servidor exclusivo para o SGBD.

Similares aos procedimentos são os gatilhos, rotinas que serão executadas sem chamada explícita, por estarem associadas aos comandos INSERT, DELETE e UPDATE.

Exercícios

1. Criar a função “ultimoPedido”, que receba o código do cliente e retorne o código do seu último pedido.
2. Criar o procedimento “pizzasDoUltimoPedido”, que recebe o código do cliente como parâmetro e lista o nome das pizzas do último pedido, que será obtido com a função “ultimoPedido”, criada anteriormente.

As respostas dos exercícios deste livro estão disponíveis para *download* no seguinte *link*: https://www.senaispeditora.com.br/downloads/respostas/principios_banco_dados_mysql_respostas.pdf.

Considerações finais

São poucos os programas que não precisam do armazenamento e recuperação de dados. Por isso, o uso de Sistemas Gerenciadores de Banco de Dados tornou-se conhecimento imprescindível para analistas e desenvolvedores.

Em um ambiente de desenvolvimento corporativo, o analista responde pelo levantamento dos requisitos com o cliente e pela criação dos modelos conceitual e lógico do banco de dados. Um especialista em banco de dados será responsável pela criação do modelo físico, *triggers*, funções e procedimentos.

No caso de um desenvolvedor autônomo, todas as tarefas serão de sua responsabilidade. Independentemente de o desenvolvedor ser o responsável por tudo ou por uma parte, é necessário reconhecer o que o outro profissional modelou. Como desenvolvedor, o profissional deve ter conhecimento de banco de dados.

Este livro abordou:

- modelagem de dados;
- comandos DDL;
- comandos DML;
- maneiras diversas de realizar consultas;
- procedimentos e funções;
- *triggers*.

Esses assuntos constituem o embasamento necessário para iniciar a carreira de desenvolvedor de banco de dados para a persistência dos dados em aplicações comerciais, industriais ou qualquer outro segmento que necessite trabalhar com a linguagem de programação.

É importante observar que qualquer linguagem de programação usa o que foi aprendido neste livro de maneira diferente. O aluno deve estudar o acesso e a manipulação de banco de dados na linguagem que pretende trabalhar e utilizar os exemplos deste livro como referência.

Então, o próximo passo é escolher uma linguagem de programação e aplicar os conhecimentos de banco de dados nela e ir além, de modo a aprofundar os estudos sobre as bibliotecas da linguagem e desenvolver sistemas completos.

Referências

ALVES, William Pereira. **Banco de dados: teoria e desenvolvimento**. São Paulo: Érica, 2009.

BELÉM, T. **Relacionamento de tabelas no MySQL**. Disponível em <<http://blog.thiagobelem.net/relacionamento-de-tabelas-no-mysql>>. Acesso em: 15 fev. 2017.

BOSON. Treinamentos em Tecnologia. **MySQL Inner Join**. Disponível em: <<http://www.bosontreinamentos.com.br/mysql/mysql-inner-join-consultar-dados-em-duas-ou-mais-tabelas-29/>>. Acesso em: 15 fev. 2017.

COLI, R. **Tipos de dados no MySQL**. Disponível em <<http://www.rcoli.com.br/2012/08/tipos-de-campos-no-mysql-saiba-como-escolher-o-tipo-correto/>>. Acesso em: 15 fev. 2017.

DAMAS, Luis. **SQL Structured Query Language**. 6. ed. Rio de Janeiro: LTC, 2007.

DATE, Chistopher J. **Introdução a Sistemas de Banco de Dados**. 8. ed. Rio de Janeiro: Campos, 2004.

DEVMEDIA – SQL JOIN. **Dica de leitura**. Disponível em <<http://www.devmedia.com.br/sql-join-entenda-como-funciona-o-retorno-dos-dados/31006>>. Acesso em: 15 fev. 2017.

ELMASRI, Ramez; NAVATHE, Shamkant. **Sistemas de banco de dados**. 6. Ed. São Paulo: Pearson, 2011.

MACHADO, Felipe Nery Rodrigues. **Projeto e implementação de banco de dados**. 2. ed. São Paulo: Érica, 2008.

MYSQL. **Manual**. Disponível em: <<https://dev.mysql.com/doc/refman/5.7/en/tutorial.html>>. Acesso em: 15 fev. 2017.

MYSQLTUTORIAL. **Tutorial básico de MySQL**. Disponível em: <<http://www.mysqltutorial.org/basic-mysql-tutorial.aspx>>. Acesso em: 15 fev. 2017.

SILVA, Robson Soares. **Oracle 10g Express Edition: Guia de Instalação, Configuração e Administração**. São Paulo: Érica, 2007.

Sobre o autor

Glauber Roberto Paschoalini é especialista em Engenharia de Componentes utilizando Java pelas Faculdades Integradas de Ourinhos e tecnólogo em Análise e Desenvolvimento de Sistemas pela Universidade Nove de Julho (Uninove). Com mais de vinte anos de experiência na área de programação, coordenou equipes com as quais projetou e desenvolveu *softwares* para indústria e comércio, utilizando as linguagens COBOL, C, CLIPPER e Java. Atuou como instrutor no SENAI-SP (Unidade Santa Cecília) nas unidades curriculares de fundamentos de programação, algoritmo, linguagem orientada a objetos, programação *mobile*, banco de dados e projeto de *software*. Ministrou o treinamento “Prática pedagógica para cursos estruturados com base em competências” para outros instrutores e participou dos comitês para atualização da grade curricular dos cursos de informática no SENAI-SP e SENAI-DN. É autor do livro *Princípios de lógica de programação*, publicado pela SENAI-SP Editora.

SENAI-SP editora

Coordenação editorial

Glauce Perusso Pereira Dias Muniz

Direitos autorais

Edilza Alves Leite

Viviane Medeiros de Souza Guedes

Edição

Paula Hercy Cardoso Craveiro

Assistência editorial

Mariane Cristina de Oliveira

Preparação

Lucimara Leal

Revisão

Breno Beneducci

Capa

Inventum Design

Ilustração

Acervo do autor

Ivan Sardella

Diagramação

Globaltec Editora

Coordenação de produção gráfica

Rafael Zemantuaskas

Produção gráfica

Ana Carolina Almeida de Moura

© SENAI-SP Editora, 2019

A SENAI-SP Editora empenhou-se em identificar e contatar todos os responsáveis pelos direitos autorais deste livro. Se porventura for constatada omissão na identificação de algum material, dispomo-nos a efetuar, futuramente, os possíveis acertos.

Esta publicação integra uma série da SENAI-SP Editora especialmente criada para apoiar os cursos do SENAI-SP.

O mercado de trabalho em permanente mudança exige que o profissional se atualize continuamente ou, em muitos casos, busque qualificações. É para esse profissional, sintonizado com a evolução tecnológica e com as inovações nos processos produtivos, que o SENAI-SP oferece muitas opções em cursos, em diferentes níveis, nas diversas áreas tecnológicas.

ISBN 978-85-8393-992-4



== **SENAI** ==